



# Agentic AI Lead Identification System

Automated lead identification for an Indian law firm. Scrapes 100% free, public sources, enriches with Jina AI + Groq LLM, deduplicates with ChromaDB RAG, and outputs to CSV + Google Sheets — zero paid APIs, zero manual work.

Quick Start

## 1. Clone and set up environment

```
cd lead-identification-system
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
```

## 2. Install dependencies

```
pip install -r requirements.txt
```

## 3. First-time setup (downloads spaCy model + Playwright browser)

```
python main.py --setup
```

## 4. Configure secrets

```
cp .env.example .env
```

**Edit .env with your Groq keys, Jina key, Google Sheets credentials**

## 5. Run on demand (last 30 days)

```
python main.py --mode manual
```

## 6. Scheduled daily run (last 7 days — for cron/GitHub Actions)

```
python main.py --mode scheduled
```

Agent Architecture

## Orchestrator

- └─ Phase 1 — Primary scrapers (all parallel)
  - | └─ RSS Scraper — 9 Indian startup news feeds + spaCy NER
  - | └─ Accelerator Scraper — YC, Antler, 100X.VC, Sequoia Surge → Jina + Groq
  - | └─ Gov Registry Scraper — SEBI AIF diff, IP India, GOI press releases
- └─ Phase 2 — Watchlist-driven scrapers (parallel)
  - | └─ MCA Scraper — Zaubra Corp via Jina + Groq, OGD bulk CSV
  - | └─ Job Scraper — Wellfound, LinkedIn Jobs, company career pages
- └─ Scoring Agent — signal weights → Priority A / B / C
- └─ RAG Dedup — ChromaDB cosine similarity + company name match
- └─ Enrichment Agent — Jina fetches team pages → Groq extracts contacts
- └─ Output Agent — CSV + JSON + Google Sheets + terminal summary

## How Jina + Groq replace brittle scraping

Every unstructured page (portfolio pages, team pages, Zaubra Corp) goes through:

URL → Jina Reader → clean markdown → Groq → structured JSON

This is layout-agnostic — no CSS selectors that break when a site redesigns. Structured sources (RSS feeds, OGD CSV) are still parsed directly — no LLM overhead needed.

## Signal Weights

### SignalWeightPriority

founder\_asking\_publicly

10

A (≥10)

legal\_job\_posted

9

A

job\_posting\_general\_counsel

9

A

sebi\_aif\_new\_fund

9

A

funding\_announced

8

A

international\_entity\_flip

8

A

mca\_share\_allotment

7

B

international\_expansion\_news

7

B

esop\_expansion

6

B

mca\_charge\_creation

6

B

cfo\_vp\_finance\_hired

5

B

mca\_authorized\_capital

5

B

accelerator\_cohort

4

C

new\_director\_mca

3

C

dpiit\_recognition

3

C

ip\_filing

3

C

trademark\_filing

2

C

new\_gst\_registration

2

C

Data Sources (All Free)

News & RSS: YourStory, Inc42, Entrackr, VCCircle, MediaNama, [Trak.in](#), OfficeChai, StartupTalky, IndianWeb2

Corporate Registries: Zauba Corp (Jina + Groq), MCA OGD bulk CSV

Government: SEBI AIF list (diff-based), IP India trademarks, DPIIT Startup India, GOI press releases

Job Boards: Wellfound (AngelList), LinkedIn Jobs (public pages), company career pages

Accelerators: Y Combinator, Antler India, [100X.VC](#), Sequoia Surge, Accel Atoms, Z Nation Lab

Multi-Account Key Rotation

Groq (LLM)

Free tier: 14,400 requests/day per account

Set up to 5 keys (GROQ\_API\_KEY through GROQ\_API\_KEY\_4)

Router automatically switches to the next key on rate limit (HTTP 429)

3 accounts = 43,200 requests/day — no practical limit for daily runs

Jina AI Reader (URL → Markdown)

Free tier: 10M tokens per account

Set up to 3 keys (JINA\_API\_KEY through JINA\_API\_KEY\_2)

Keys rotate round-robin on every fetch

At ~3,000 tokens/page and ~80 pages/run: 3 accounts = 1,000+ daily runs before top-up

Email Enrichment (Free Stack)

Founder emails are discovered through a 4-step free pipeline (stops at first good hit):

Company website scrape — /team, /about, /contact → LLM extracts named business emails

LinkedIn public profile — site:linkedin.com/in "<founder>" "<company>" (no login)

[Skrapp.io](https://skrapp.io) — free email finder (150/month). Only emails with score > 70 are accepted

Domain pattern + SMTP MX check — last resort; only patterns that pass an SMTP RCPT (250) check

Strict rules: personal domains (@gmail, @yahoo, @hotmail, @outlook, @rediffmail) are never accepted; emails whose domain doesn't match the company website are rejected; generic mailboxes (info@, support@, contact@) are used only as a last resort. Every email records its enrichment\_source.

Skrapp setup (optional)

Sign up at [skrapp.io](https://skrapp.io) (free tier = 150 emails/month)

Settings → API → copy your access key

Set SKRAPP\_API\_KEY in .env

If left blank, the pipeline simply skips step 3 and falls back to website / LinkedIn / SMTP.

Verification Stack (Free, No Apollo)

Before any lead is written to the sheet it must pass:

ICP hard gate — blocklist (SpaceX, ISRO, DRDO, ministries, MNCs, PSUs...) auto-rejects instantly; everything else gets an LLM "is this an Indian startup?" check. Rejected leads are written to leads\_rejected\_{date}.csv with a reason.

Company existence — checked in order against Razorpay Business Directory → InstaFinancials → [CompanyDetails.in](https://CompanyDetails.in) → CompanySeekers (first hit wins).

Funding claims — corroborated against ≥ 2 public news sources (1 source → low confidence).

Output files: leads\_verified\_{date}.csv (verified=true) · leads\_unverified\_{date}.csv (manual review) · leads\_rejected\_{date}.csv (ICP/blocklist failures). The Google Sheet receives only verified leads that have a populated founder\_name.

Google Sheets Output

The system auto-appends new leads to a shared Google Sheet — accessible from any browser with no technical setup for the legal team.

[Google Cloud Console](https://console.cloud.google.com) → Create project → Enable Sheets API + Drive API

Create a Service Account → download JSON key file

Share your Google Sheet with the service account email (Editor permission)

Set GOOGLE\_SERVICE\_ACCOUNT\_JSON (path to JSON file) and GOOGLE\_SHEET\_ID in .env

GitHub Actions (Daily Automation)

Runs every day at 8:00 AM IST via `.github/workflows/daily_leads.yml`. Also triggerable manually from GitHub UI (Acts as "Get Leads Now" button).

Add these secrets in GitHub → repo → Settings → Secrets and variables → Actions:

SecretDescription

GROQ\_API\_KEY

Primary Groq key

GROQ\_API\_KEY\_1 — GROQ\_API\_KEY\_4

Additional Groq accounts for rotation

SKRAPP\_API\_KEY

(Optional) [Skrapp.io](https://skrapp.io) email-finder key, free tier 150/month

GOOGLE\_SERVICE\_ACCOUNT\_JSON

Full contents of your service account JSON

GOOGLE\_SHEET\_ID

ID from your Google Sheet URL

Query Your Lead Database

`python main.py --query "Show me all Series A leads from last 2 weeks"`

`python main.py --query "Which companies are hiring a Head of Legal?"`

`python main.py --query "All Priority A leads in Bangalore"`

`python main.py --query "Companies that raised funding and have no legal hire yet"`

Tech Stack

LayerToolNotes

Language

Python 3.11+

RSS Parsing

feedparser

Structured XML — no LLM needed

Page fetching

Jina AI Reader ([r.jina.ai](https://r.jina.ai))

Converts any URL to clean markdown

JS pages

playwright

For pages Jina can't reach

LLM extraction

Groq (Llama 3.3 70B)

Classifies signals, extracts contacts

NLP / NER

spaCy (`en_core_web_sm`)

Company/person name extraction from RSS

Vector DB

chromadb (local)

Dedup + semantic search

Embeddings

sentence-transformers

Runs on CPU, no GPU needed

Google Sheets

gsread  
Output for legal team  
Scheduling  
GitHub Actions  
Free, 2000 min/month  
Data  
pandas  
OGD bulk CSV processing  
Design Principles

Each agent is a pure async function — no side effects, no disk writes inside agents  
Orchestrator owns all I/O — agents return dicts, orchestrator decides what to store  
RAG is the single source of truth — ChromaDB prevents the same company appearing from multiple sources  
LLM extracts, never invents — Groq only processes already-scraped text; hallucination is structurally impossible  
Graceful degradation — one broken scraper never kills the pipeline; errors are logged and skipped  
All secrets in .env — never hardcoded, never in .env.example

this is my project [readme.md](#) i want to change my entire approach and Crawl4AI  
what should i change? give the list? tell me if i should send any code to you

Yes — if you are moving your pipeline to **Crawl4AI-first**, you should change more than just the scraper layer. Your current README is built around a mixed strategy of RSS parsing + Jina Reader + Groq for page normalization, but Crawl4AI already gives you browser automation, markdown generation, filtering, and both CSS/XPath and LLM extraction, so the cleaner approach is to redesign the ingestion and extraction layers around it. <sup>[1]</sup> <sup>[2]</sup>

## What to change

You should replace this part of the architecture first:

| Current piece                            | Change to                                                                                                         | Why                                                                                                                                                          |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jina Reader as universal page normalizer | Crawl4AI markdown / fit-markdown pipeline                                                                         | Crawl4AI already produces clean markdown and filtered markdown designed for LAG/RAG and LLM ingestion. <sup>[2]</sup> <sup>[1]</sup>                         |
| Jina + Groq for all unstructured pages   | Crawl4AI extraction strategy split: CSS/XPath for structured pages, LLM extraction only for genuinely messy pages | Crawl4AI docs explicitly support CSS/XPath extraction for structured content and LLM extraction for unstructured content. <sup>[1]</sup>                     |
| Playwright as fallback browser           | Crawl4AI as the primary browser crawl layer                                                                       | Crawl4AI includes browser-level crawling controls such as page timeout, full-page scanning, delays, and dynamic page handling. <sup>[1]</sup> <sup>[2]</sup> |
| Ad hoc parsing by source type            | Per-source Crawl4AI configs and schemas                                                                           | Crawl4AI is designed to run with crawler config files and extraction schemas, which fits source-specific scrapers better. <sup>[1]</sup>                     |

| Current piece                                              | Change to                                        | Why                                                                                                        |
|------------------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| “LLM extracts, never invents” via Groq after Jina markdown | “Crawl → filter → structured extract → validate” | This reduces hops and lets you isolate extraction errors by stage. <a href="#">[1]</a> <a href="#">[2]</a> |

## Architecture rewrite

I’d suggest rewriting your agent architecture to something like this:

1. **Discovery layer:** RSS feeds, public registries, accelerator/company lists, sitemap seeds, search seeds.
2. **Crawl layer:** Crawl4AI handles fetch, JS rendering, scrolling/full-page scan, markdown generation, and optional filtering. [\[2\]](#) [\[1\]](#)
3. **Extraction layer:**
  - CSS/XPath schemas for repeatable pages like job cards, portfolio grids, registry tables. [\[1\]](#)
  - LLM extraction only for founder/team/contact pages, irregular press pages, or free-form announcements. [\[1\]](#)
4. **Normalization layer:** canonical company identity, source URL, timestamp, signal type, confidence, evidence text.
5. **Dedup/entity resolution:** ChromaDB only if you truly need semantic clustering; otherwise start with deterministic dedup plus embeddings for edge cases.
6. **Verification layer:** existence check, funding corroboration, ICP gate.
7. **Output layer:** CSV, Sheets, query interface.

That change matters because Crawl4AI is strongest as a **crawl + extract orchestration layer**, not just a markdown converter. [\[2\]](#) [\[1\]](#)

## Specific README edits

Your README currently over-emphasizes Jina and Groq. I would change these sections:

- **Project title/subtitle:** remove “enriches with Jina AI + Groq LLM” and replace with “crawls public web sources with Crawl4AI, extracts structured signals, deduplicates, verifies, and exports.”
- **Quick Start:** replace any setup that is only there for Jina Reader usage with Crawl4AI install/setup instructions.
- **Agent Architecture:** rename “Primary scrapers” to “Source discovery + Crawl4AI extraction pipelines.”
- **How Jina + Groq replace brittle scraping:** replace this entire section with “How Crawl4AI reduces brittle scraping.”
- **Tech Stack:** replace “Page fetching = Jina AI Reader” with “Crawl/extraction = Crawl4AI”; keep Playwright only if you directly use it outside Crawl4AI for edge sites.

- **Key rotation:** your Groq rotation section should shrink, because if Crawl4AI handles CSS/XPath extraction for many pages, LLM volume drops. <sup>[1]</sup>
- **Email enrichment:** this can stay, but step 1 should become Crawl4AI-powered page extraction from /team, /about, /contact.
- **Design principles:** add “schema-first extraction” and “LLM only when selectors fail or the page is semantically irregular.”

## What should change in code

At code level, I'd recommend these changes:

- Create a `crawl_service.py` that wraps Crawl4AI's async crawler and standard run config.
- Create source-specific extraction schemas:
  - `schemas/jobs_css.json`
  - `schemas/accelerators_css.json`
  - `schemas/registry_xpath.json`
- Create two extraction paths:
  - `extract_structured()` → CSS/XPath schema
  - `extract_unstructured()` → LLM strategy over markdown/fit-markdown
- Standardize every result into one model, for example:
  - `company_name`
  - `source_type`
  - `source_url`
  - `signal_type`
  - `signal_strength`
  - `evidence_text`
  - `event_date`
  - `contacts`
  - `confidence`
- Add content filtering before LLM extraction, because Crawl4AI supports filtered markdown outputs for better relevance and token efficiency. <sup>[1]</sup>
- Add crawl configs per source for:
  - `scan_full_page`
  - `delay_before_return_html`
  - `wait_until`
  - `css_selector`
  - cache mode bypass for fresh runs. <sup>[1]</sup>

## What to remove or reduce

You should probably remove or downgrade these ideas:

- **Jina as the default fetch path** — redundant if Crawl4AI is your primary crawler. <sup>[2]</sup>
- **Groq for every messy page** — too expensive in complexity and error surface; use only for free-form extraction. <sup>[1]</sup>
- **“Hallucination is structurally impossible”** — I would soften this claim; LLM extraction is still probabilistic even if constrained by scraped text.
- **Heavy RAG-first dedup messaging** — for lead systems, exact-name/domain normalization plus fuzzy matching usually beats semantic dedup as the first pass.

## Best new positioning

A stronger README message would be:

- Crawl4AI handles crawling, rendering, markdown generation, filtering, and schema-based extraction. <sup>[2]</sup> <sup>[1]</sup>
- Deterministic extraction is preferred for stable sources; LLM extraction is reserved for irregular pages. <sup>[1]</sup>
- Verification happens after extraction, not during extraction.
- ChromaDB is optional or secondary, not the centerpiece.

## My recommendation

Your new stack should be:

- **Crawl4AI** for crawl/render/extract. <sup>[2]</sup> <sup>[1]</sup>
- **Groq** only for selected unstructured extraction or classification.
- **spaCy** only where RSS/title/entity spotting is still useful.
- **ChromaDB** only for ambiguous dedup or natural-language querying, not as the main dedup engine.
- **Google Sheets + CSV** can stay unchanged.

Yes, send code if you want concrete changes. The most useful files would be:

- `main.py`
- your current scraper modules
- any Jina wrapper/service
- Groq extraction code
- dedup logic
- output/Sheets integration
- `current README.md`

Best order to send them:

- 1. README.md
- 2. main.py
- 3. one representative scraper using Jina/Groq
- 4. your data model/schema files
- 5. dedup + output modules

I can then give you a **file-by-file migration plan**, or directly rewrite the README and propose a new project structure around Crawl4AI.



## this is the existing code structure

tell me what i should change and what i should not to implement the crawl4AI into this  
also tell me if you need any content of the codes

Based on the structure in your screenshots, you **do not** need to rewrite the whole project to adopt Crawl4AI. The best path is to replace the fetch/extraction layer, standardize scraper outputs, and keep your orchestration, scoring, verification, config, and output pipeline mostly intact. [\[16\]](#) [\[17\]](#) [\[18\]](#)

### Keep vs change

Here’s the practical split:

| Keep                                                                         | Change                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| agents/orchestrator.py                                                       | utils/jina_fetch.py should become a Crawl4AI wrapper or be removed. <a href="#">[17]</a>                                                                                                                                                     |
| agents/scoring_agent.py                                                      | Scraper internals inside rss_scraper.py, accelerator_scraper.py, job_scraper.py, mca_scraper.py, gov_registry_scraper.py, and similar files should switch to Crawl4AI-based extraction where pages are HTML/web-driven. <a href="#">[18]</a> |
| agents/output_agent.py                                                       | utils/http_fetch.py should be reduced to a lightweight fallback only if some sources are plain APIs/files. <a href="#">[17]</a>                                                                                                              |
| config/keywords.py, signal_weights.py, sources.py, blocklist.py, settings.py | Any logic tightly coupled to “Jina markdown → Groq JSON” assumptions should be refactored into a generic extraction interface. <a href="#">[17]</a> <a href="#">[18]</a>                                                                     |
| agents/icp_gate.py, verifier.py                                              | llm_router.py / utils/llm_router.py should stop being mandatory in every scraper and become optional for only messy pages. <a href="#">[16]</a> <a href="#">[17]</a>                                                                         |
| rag/embedder.py, rag/vector_store.py                                         | Dedup should happen after normalized extraction, not mixed inside scraper logic if that’s currently happening. <a href="#">[17]</a>                                                                                                          |
| main.py, README.md, requirements.txt                                         | Apollo-specific code should probably be removed if your new promise is fully free/public-source based. <a href="#">[18]</a>                                                                                                                  |

## What to change first

Start with the lowest-risk integration path:

1. Add a new module like `utils/crawl4ai_fetch.py`.
2. Define one common return shape for crawled pages, for example:
  - `url`
  - `raw_html`
  - `markdown`
  - `fit_markdown`
  - `metadata`
  - `links`
  - `status_code`
  - `source_name`
3. Update one scraper only, ideally `accelerator_scraper.py` or `job_scraper.py`, to use this new wrapper first. <sup>[18]</sup>
4. Once that works, migrate the rest of the HTML-heavy scrapers one by one.

This is safer than replacing all scraper logic at once. <sup>[18]</sup>

## What not to change yet

Do not change these early unless the code forces you to:

- Your scoring system and signal weights.
- Your config folder structure.
- Your CSV/Google Sheets output flow.
- Your verification gates.
- Your ChromaDB layer, unless dedup is currently entangled with raw page fetching.
- Your CLI contract in `main.py` such as `--setup`, `--mode manual`, `--mode scheduled`, unless setup currently depends on Jina-specific bootstrapping. <sup>[17]</sup> <sup>[16]</sup> <sup>[18]</sup>

That means Crawl4AI should enter as an **infrastructure replacement**, not as a full product redesign in phase 1. <sup>[17]</sup> <sup>[18]</sup>

## Recommended file-level plan

Given your tree, I'd map the migration like this:

## High priority changes

- `utils/jina_fetch.py` → replace with `utils/crawl4ai_fetch.py` or rewrite the file behind the same interface. <sup>[17]</sup>
- `utils/http_fetch.py` → keep only for direct API/CSV/JSON downloads. <sup>[17]</sup>
- `agents/accelerator_scraper.py` → strong candidate for Crawl4AI because portfolio/cohort pages are often semi-structured HTML. <sup>[18]</sup>
- `agents/job_scraper.py` → strong candidate because public jobs pages often need rendering + structured extraction. <sup>[18]</sup>
- `agents/mca_scraper.py` → use Crawl4AI only if you are actually crawling HTML pages; keep direct file/CSV parsing for bulk datasets. <sup>[18]</sup>
- `agents/enrichment_agent.py` → make it consume normalized crawl outputs instead of Jina-specific text blobs. <sup>[18]</sup>

## Medium priority changes

- `llm_router.py` and `utils/llm_router.py` → keep, but make them route only when deterministic extraction fails. <sup>[16]</sup> <sup>[17]</sup>
- `utils/llm_json.py` → keep if it validates schema outputs from Groq or other LLM extraction. <sup>[17]</sup>
- `agents/parser_filter.py` → probably rename or repurpose into normalization/cleaning after Crawl4AI extraction. <sup>[18]</sup>

## Low priority / likely unchanged

- `config/*` files. <sup>[17]</sup>
- `rag/*` files. <sup>[17]</sup>
- `agents/scoring_agent.py`, `output_agent.py`, `icp_gate.py`, `verifier.py`. <sup>[18]</sup>
- `main.py` except wiring and setup commands. <sup>[16]</sup>

## Best integration pattern

You have two good implementation options:

### Option A: Minimal-change

Keep scraper APIs unchanged and only replace the internal fetcher.

Example idea:

- `accelerator_scraper.py` still calls something like `fetch_page(url)`
- but under the hood it now uses Crawl4AI instead of Jina

Use this if your project is already stable and you want the fastest migration. <sup>[17]</sup> <sup>[18]</sup>

## Option B: Cleaner architecture

Introduce three layers:

- `crawl_service`
- `extract_service`
- `normalize_service`

Then scrapers become thin source coordinators instead of doing everything. This is the better long-term design if your current scrapers are large and mixed with fetch + parse + classify logic. <sup>[18]</sup>

For your project, I'd recommend **Option A first, then Option B gradually**. <sup>[17]</sup> <sup>[18]</sup>

## What I need from you

Yes — to give you an accurate migration plan, I need some code contents. The most useful files are:

1. `utils/jina_fetch.py`
2. `utils/http_fetch.py`
3. `agents/accelerator_scraper.py`
4. `agents/job_scraper.py`
5. `agents/orchestrator.py`
6. `llm_router.py` or `utils/llm_router.py`
7. `agents/enrichment_agent.py`
8. `main.py`

If you want the **minimum necessary send**, start with just these four:

- `utils/jina_fetch.py`
- `agents/accelerator_scraper.py`
- `agents/orchestrator.py`
- `main.py`

With those, I can tell you:

- exactly what to replace,
- what interfaces to preserve,
- where Crawl4AI should enter,
- and whether you should keep Groq in the loop or not.

Paste those files one by one, starting with `utils/jina_fetch.py`.

# give me the entire ideation in one go to understand and build

Yes — here's the **full ideation in one go**: how to rethink your existing lead-identification system around Crawl4AI, what stays, what changes, what the final architecture should look like, and how to build it step by step. Crawl4AI already gives you async crawling, browser configuration, HTML-to-markdown, filtered “fit\_markdown,” and both deterministic and LLM extraction strategies, so the cleanest redesign is to make it your primary crawl-and-extract engine instead of using Jina as the universal page normalizer.<sup>[19] [20] [21]</sup>

## Core shift

Your current mental model is roughly:

source → fetch page → Jina markdown → Groq extraction → normalize → score → dedup → verify → output

The Crawl4AI-first model should become:

source discovery → Crawl4AI crawl/render → deterministic extraction where possible → LLM extraction only where needed → normalize → score → dedup → verify → output<sup>[22] [20] [19]</sup>

That change matters because Crawl4AI supports:

- browser-based async crawling through AsyncWebCrawler, BrowserConfig, and CrawlerRunConfig,<sup>[23] [19]</sup>
- markdown plus filtered fit\_markdown for LLM-friendly processing,<sup>[24] [25] [21]</sup>
- CSS/XPath extraction for stable page layouts,<sup>[26] [22]</sup>
- LLM extraction over markdown or fit\_markdown for messy pages.<sup>[27] [20]</sup>

## What remains

A lot of your project should stay.

Keep these ideas and modules conceptually:

- main.py as the CLI entry point.
- agents/orchestrator.py as the workflow controller.
- scoring\_agent.py and your signal weights.
- icp\_gate.py, verifier.py, and blocklist logic.
- output\_agent.py for CSV/Sheets/JSON.
- config/ for sources, keywords, settings, and signal definitions.
- rag/ for query + dedup support if you still want semantic search.<sup>[28] [29] [30]</sup>

These are business-logic layers, not crawl-engine layers, so they should not be rewritten just because the fetcher changes.<sup>[29] [30]</sup>

## What changes

The biggest rewrite is the ingestion/extraction boundary.

## Replace this

- `utils/jina_fetch.py` as the default path for turning URLs into LLM-ready content. <sup>[29]</sup>

## Add this

- `utils/crawl4ai_fetch.py`
- `services/crawl_service.py`
- `services/extract_service.py`
- `services/normalize_service.py`

You do not strictly need new folders, but this separation will make your project easier to maintain. Crawl4AI has first-class support for crawl configuration, content filtering, markdown generation, and extraction strategies, so it fits naturally into a service layer rather than being embedded ad hoc in every scraper. <sup>[20]</sup> <sup>[21]</sup> <sup>[19]</sup>

## New architecture

A clean architecture for your system would be:

### 1. Discovery layer

This is how you find candidate URLs or records.

Sources:

- RSS feeds
- accelerator/cohort pages
- registry pages and bulk files
- job boards
- company/team/about/contact pages
- press releases and government sites

This layer should only answer: **what should be crawled?**

### 2. Crawl layer

This layer uses Crawl4AI to fetch and render pages.

It should return a standard object like:

- `url`
- `status_code`
- `html`

- `cleaned_html`
- `markdown`
- `fit_markdown`
- `links`
- `metadata`
- `source_name`
- `crawl_timestamp`

Crawl4AI supports browser-level config such as `headless`, `browser_type`, JavaScript enabled state, timeouts, image waiting, `css_selector`, and excluded tags, which is exactly what you need for mixed public-source crawling. [\[19\]](#) [\[23\]](#)

### 3. Extraction layer

This is the heart of the redesign.

Use **three extraction modes**:

#### 1. Direct parse mode

For RSS/XML/CSV/JSON/OGD sources, do not use Crawl4AI extraction or LLMs. Parse directly.

#### 2. Deterministic extraction mode

For stable HTML structures, use:

- `JsonCssExtractionStrategy` for repetitive cards/lists/tables, [\[22\]](#)
- `JsonXPathExtractionStrategy` for messy or deeply nested DOMs. [\[31\]](#) [\[22\]](#)

#### 3. LLM extraction mode

For messy, human-written pages, run LLM extraction on:

- `markdown` for broader context, or
- `fit_markdown` for cleaner, token-efficient content. [\[21\]](#) [\[27\]](#) [\[20\]](#)

The rule should be: **deterministic first, LLM second**. Crawl4AI's no-LLM extraction strategies are designed to be fast and predictable for production use on structured pages. [\[26\]](#) [\[22\]](#)

### 4. Normalization layer

Every scraper should emit the same data model regardless of source.

Suggested schema:

- `company_name`
- `canonical_company_name`
- `company_domain`
- `source_name`
- `source_type`

- source\_url
- discovered\_at
- event\_date
- signal\_type
- signal\_weight
- priority
- evidence\_text
- evidence\_url
- location
- founder\_name
- founder\_role
- founder\_email
- enrichment\_source
- verified
- verification\_reason
- confidence
- raw\_extraction\_mode

This is where your business logic becomes consistent.

## 5. Scoring layer

Your signal weights can remain mostly unchanged:

- funding news
- legal hiring
- founder asking publicly
- AIF activity
- share allotment
- international expansion
- ESOP growth
- director addition
- IP/trademark signals

Scoring should work only on normalized records, not on raw HTML fragments.

## 6. Dedup layer

Use a layered strategy:

1. exact domain match,
2. normalized company-name match,
3. fuzzy match,
4. embedding similarity only for ambiguous cases.

ChromaDB should become a **secondary resolver**, not the first dedup step. Semantic dedup is useful, but lead systems usually benefit more from deterministic identity resolution first.

## 7. Verification layer

Keep your current verification stack idea:

- ICP hard gate,
- company existence verification,
- funding corroboration,
- optional founder/contact validation,
- verified vs unverified vs rejected outputs.

This should run after normalized extraction, not during extraction.

## 8. Output layer

Keep:

- CSV,
- JSON,
- Google Sheets,
- terminal summary,
- query CLI if you already have it.

## File-by-file thinking

Based on your screenshots, here is the safest mapping. <sup>[30]</sup> <sup>[28]</sup> <sup>[29]</sup>

## Keep mostly as-is

- `main.py` <sup>[28]</sup>
- `agents/orchestrator.py` <sup>[30]</sup>
- `agents/scoring_agent.py` <sup>[30]</sup>
- `agents/output_agent.py` <sup>[30]</sup>
- `agents/icp_gate.py` <sup>[30]</sup>

- `agents/verifier.py / verifier.py` <sup>[29]</sup> <sup>[30]</sup>
- `config/settings.py` <sup>[29]</sup>
- `config/signal_weights.py` <sup>[29]</sup>
- `config/sources.py` <sup>[29]</sup>
- `config/blocklist.py` <sup>[29]</sup>
- `rag/embedder.py, rag/vector_store.py` <sup>[29]</sup>

## Change heavily

- `utils/jina_fetch.py` → replace with Crawl4AI wrapper. <sup>[29]</sup>
- `utils/http_fetch.py` → keep only for raw files and APIs. <sup>[29]</sup>
- `agents/accelerator_scraper.py` → Crawl4AI deterministic/LLM hybrid. <sup>[30]</sup>
- `agents/job_scraper.py` → Crawl4AI with wait conditions/selectors. <sup>[30]</sup>
- `agents/gov_registry_scraper.py` → mixed mode: direct parse for files, Crawl4AI only for web pages. <sup>[30]</sup>
- `agents/mca_scraper.py` → direct parse for official datasets; Crawl4AI only for HTML surfaces. <sup>[30]</sup>
- `agents/enrichment_agent.py` → use Crawl4AI for `/team`, `/about`, `/contact`. <sup>[30]</sup>
- `llm_router.py` and `utils/llm_router.py` → make optional, not mandatory. <sup>[28]</sup> <sup>[29]</sup>

## Remove or reconsider

- `agents/apollo_enricher.py` if you want the project to remain “100% free/public sources.” <sup>[30]</sup>
- any logic that assumes “URL always goes through Jina first.”
- any claim that LLM extraction is fully hallucination-proof.

## Source strategy

Do not use one strategy for every source.

### RSS/news

Use direct parsing first:

- parse feed items,
- extract company hints with spaCy if useful,
- then Crawl4AI only on shortlisted article URLs for full evidence extraction.

## Accelerator pages

Great fit for Crawl4AI deterministic extraction:

- portfolio cards,
- cohort tables,
- company links,
- founder names where visible.

Use CSS schema first. If pages vary too much, use XPath. [\[31\]](#) [\[22\]](#)

## Job boards and career pages

Good fit for Crawl4AI browser mode because some pages are JS-heavy.

Use:

- wait conditions,
- optional JavaScript,
- selectors for job rows/cards,
- fallback LLM extraction only for messy text pages. [\[32\]](#) [\[23\]](#) [\[31\]](#)

## Government and bulk registries

Do not use Crawl4AI when the source is already:

- CSV,
- JSON,
- XML,
- downloadable tables.

Use direct parsers. Crawl4AI is for public web content, not for replacing clean data pipelines.

## Team/about/contact pages

Use Crawl4AI crawl + markdown/fit\_markdown + targeted LLM extraction to get:

- founder name,
- leadership role,
- business email found on-site,
- evidence snippet.

fit\_markdown is especially useful here because it strips boilerplate and keeps the main content.

[\[25\]](#) [\[24\]](#) [\[21\]](#)

## Recommended extraction policy

Use this decision tree for every page:

1. Is the source already structured like CSV/XML/JSON?  
→ Parse directly.
2. Is the page a repeatable list/grid/table?  
→ Use CSS extraction schema. <sup>[22]</sup>
3. Is the DOM awkward or attributes/paths are tricky?  
→ Use XPath extraction schema. <sup>[31] [22]</sup>
4. Is the page unstructured prose, mixed biographies, or narrative announcements?  
→ Use LLM extraction on `fit_markdown`. <sup>[27] [20] [21]</sup>

This one rule will save you a lot of time and cost.

## How Crawl4AI should be used

A practical Crawl4AI wrapper should support:

- browser config,
- per-source crawl config,
- markdown generation,
- fit-markdown generation,
- extraction strategy injection,
- retries,
- logging,
- cache mode.

Crawl4AI docs show the core pattern:

- define `BrowserConfig`,
- define `CrawlerRunConfig`,
- call `AsyncWebCrawler().arun(url, config=run_config)`. <sup>[23] [19]</sup>

For dynamic pages you can control things like:

- `page_timeout`,
- `wait_for_images`,
- `css_selector`,
- `excluded_tags`,
- JavaScript execution,
- cache behavior. <sup>[19] [23] [31]</sup>

## Minimal module design

Here's the clean project idea.

`services/crawl_service.py`

Responsibilities:

- single-page crawl,
- batch crawl,
- source-specific run configs,
- retries and backoff,
- result object standardization.

`services/extract_service.py`

Responsibilities:

- `extract_via_css(schema, crawl_result)`
- `extract_via_xpath(schema, crawl_result)`
- `extract_via_llm(prompt, crawl_result, input_format="fit_markdown")`

Crawl4AI extraction strategies operate on different stages such as `html`, `markdown`, `fit_html`, and `fit_markdown`, so this service should choose the correct input format explicitly. <sup>[20]</sup>

`services/normalize_service.py`

Responsibilities:

- normalize names/domains,
- unify output records,
- attach source metadata,
- map extracted fields to your canonical lead schema.

`services/entity_service.py`

Responsibilities:

- deterministic dedup,
- fuzzy match,
- optional ChromaDB similarity.

`services/verify_service.py`

Responsibilities:

- ICP gate,
- existence check,
- funding corroboration,
- email acceptance rules,
- final verified/unverified/rejected split.

## What to do with Groq

Do not remove Groq completely unless you want a fully deterministic-only system.

Instead, reduce Groq's role to:

- classification on messy human text,
- extracting structured fields from `fit_markdown`,
- resolving ambiguous contact snippets,
- maybe “is this an Indian startup?” checks if your ICP gate still needs an LLM.

Do not send every page to Groq.

Crawl4AI's deterministic strategies are specifically meant to avoid unnecessary LLM calls for structured pages. <sup>[26]</sup> <sup>[22]</sup>

## What to do with Jina

If you are serious about Crawl4AI-first architecture, Jina should no longer be the default content fetcher. Crawl4AI already provides markdown generation and filtered markdown outputs, so using Jina for the same normalization role adds another dependency and another failure mode. <sup>[24]</sup> <sup>[21]</sup> <sup>[19]</sup>

You can:

- remove Jina entirely, or
- keep it as a temporary fallback during migration.

But the long-term clean architecture is Crawl4AI primary, no Jina dependency.

## What to do with Playwright

Crawl4AI already runs browser-based crawling, so standalone Playwright should only remain if:

- you need a very custom interaction flow,
- authentication or multi-step browser actions become necessary,
- or some sites need direct Playwright scripting outside the crawler lifecycle.

Otherwise, collapse Playwright into the Crawl4AI layer.

## Build phases

### Phase 1: Infrastructure swap

Goal: replace Jina, keep business logic intact.

Tasks:

- add Crawl4AI dependency,
- create `crawl_service.py`,
- replace `utils/jina_fetch.py`,
- migrate one scraper first,
- preserve current orchestrator and output flow.

### Phase 2: Source-by-source migration

Goal: move each scraper to the right extraction mode.

Order:

1. accelerator scraper,
2. job scraper,
3. enrichment agent,
4. government/registry web pages,
5. everything else.

### Phase 3: Normalize and dedup cleanup

Goal: make all scrapers emit one schema.

- tighten canonical company identity,
- simplify ChromaDB role,
- log extraction mode and confidence.

### Phase 4: README and product messaging

Goal: update the project story.

- replace Jina-centric wording,
- describe Crawl4AI-based pipeline,
- explain deterministic-first extraction,
- clarify LLM is used selectively.

## Suggested final README message

Your README should say something like:

- Public lead sources are discovered from RSS, accelerators, job boards, registries, and government pages.
- Crawl4AI handles rendering, markdown generation, filtered content extraction, and schema-based scraping from public web pages. [\[24\]](#) [\[19\]](#) [\[22\]](#)
- Deterministic CSS/XPath extraction is used for stable layouts; LLM extraction is reserved for irregular pages. [\[20\]](#) [\[22\]](#)
- All extracted records are normalized, scored, deduplicated, verified, and exported to CSV and Google Sheets.

That is much stronger and cleaner than “Jina + Groq replace brittle scraping.”

## Data model to build around

Make every scraper emit a list of records with this exact shape:

```
LeadRecord = {
    "company_name": str,
    "canonical_company_name": str | None,
    "company_domain": str | None,
    "source_name": str,
    "source_type": str,
    "source_url": str,
    "discovered_at": str,
    "event_date": str | None,
    "signal_type": str,
    "signal_weight": int,
    "priority": str | None,
    "evidence_text": str,
    "evidence_url": str,
    "location": str | None,
    "founder_name": str | None,
    "founder_role": str | None,
    "founder_email": str | None,
    "enrichment_source": str | None,
    "confidence": float | None,
    "verified": bool | None,
    "verification_reason": str | None,
    "raw_extraction_mode": str
}
```

The field `raw_extraction_mode` is useful for debugging:

- `direct_parse`
- `crawl4ai_css`
- `crawl4ai_xpath`
- `crawl4ai_llm_fit_markdown`

- `crawl4ai_llm_markdown`

## Engineering rules

Use these rules while building:

- One source, one scraper, one normalized output contract.
- Do not mix crawling, extraction, scoring, and verification in one file.
- Prefer deterministic extraction over LLM extraction.
- Prefer direct file parsing over browser crawling when source data is already structured.
- Use `fit_markdown` for noisy text-heavy pages.
- LLM extracts only from already-fetched content.
- The orchestrator owns I/O and persistence.
- Each agent returns data, not side effects.

## Practical build order

Build in this order:

1. `crawl_service.py`
2. one Crawl4AI wrapper function for single URL crawl
3. one deterministic extraction helper
4. one LLM extraction helper
5. migrate `accelerator_scraper.py`
6. migrate `job_scraper.py`
7. migrate `enrichment_agent.py`
8. normalize all records into one schema
9. test dedup + verify
10. update README

## Biggest mistakes to avoid

- Replacing every parser with Crawl4AI even when the source is already CSV/XML/JSON.
- Sending every page to an LLM.
- Keeping Jina and Crawl4AI both as core fetchers for too long.
- Letting each scraper invent its own output shape.
- Doing semantic dedup before exact domain/name matching.
- Coupling verification to crawl-time logic.

Exact answer to your question

What you should understand before building is this:

You are **not** building “the same Jina pipeline with Crawl4AI swapped in.”  
You are building a **deterministic-first extraction system with Crawl4AI as the core crawler**, where:

- structured sources are parsed directly,
- repeatable HTML is extracted via CSS/XPath schemas, <sup>[22]</sup>
- messy pages are turned into `fit_markdown` and then sent to LLM only when required, <sup>[21] [24] [20]</sup>
- and all downstream logic stays centered on normalized lead records.

If you want the next step, send your code in this order:

1. `utils/jina_fetch.py`
2. `agents/accelerator_scraper.py`
3. `agents/job_scraper.py`
4. `agents/orchestrator.py`
5. `main.py`

I'll then convert this ideation into a **concrete file-by-file migration plan**.

✱✱

if we were to use Crawl4AI and serpAPI approach in our projects

what would be the cost estimates and api calls limit  
and also mention groq's information

If you use **Crawl4AI + SerpAPI + Groq**, your cost profile is usually: Crawl4AI has no license fee but you still pay for your own compute/network, SerpAPI is the main fixed paid cost, and Groq can stay free at small-to-moderate usage if you stay within the model-specific rate limits. <sup>[37] [38] [39]</sup>

Pricing snapshot

| Tool     | Base cost                                                                      | Free tier / included usage                                                 | Main limit type                                                       |
|----------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Crawl4AI | No software license fee; self-hosted cost only <sup>[37]</sup>                 | Open-source, so no platform usage fee by default <sup>[37]</sup>           | Your own server/CPU/RAM/bandwidth, not vendor credits <sup>[40]</sup> |
| SerpAPI  | \$25/month Starter, \$75/month Developer, \$275/month Big Data <sup>[39]</sup> | Free plan: 250 searches/month, 50 searches/hour throughput <sup>[39]</sup> | Successful searches per month + hourly throughput <sup>[39]</sup>     |

| Tool | Base cost                                                     | Free tier / included usage                                                                                                   | Main limit type                                 |
|------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Groq | Can be \$0 on free tier depending on workload <sup>[38]</sup> | Model-specific free limits such as 14.4K RPD for llama-3.1-8b-instant and 1K RPD for llama-3.3-70b-versatile <sup>[38]</sup> | RPM, RPD, TPM, TPD at org level <sup>[38]</sup> |

## SerpAPI limits

SerpAPI's public pricing page currently shows these standard plans: Free with 250 searches/month and 50 throughput/hour, Starter at \$25/month for 1,000 searches and 200/hour, Developer at \$75/month for 5,000 searches and 1,000/hour, and Big Data at \$275/month for 30,000 searches and 6,000/hour. <sup>[39]</sup>

A “search” is counted only when it succeeds, and cached, errored, or failed searches do not count toward the monthly total; one request counts as one search regardless of whether the response has many results or an empty result set. <sup>[39]</sup>

## Rough monthly SerpAPI cost examples

| Usage pattern                                           | Estimated plan | Monthly cost                |
|---------------------------------------------------------|----------------|-----------------------------|
| Light testing, prototypes, demo runs up to 250 searches | Free           | \$0 <sup>[39]</sup>         |
| Small student/hackathon build, about 1,000 searches     | Starter        | \$25/month <sup>[39]</sup>  |
| Medium pipeline, about 5,000 searches                   | Developer      | \$75/month <sup>[39]</sup>  |
| Larger daily monitoring, about 30,000 searches          | Big Data       | \$275/month <sup>[39]</sup> |

For a lead system, SerpAPI cost depends heavily on how often you search for new companies, founders, job pages, or contact clues; if you use it only for discovery and not every downstream verification step, cost stays much lower. <sup>[39]</sup>

## Groq limits

Groq measures rate limits by RPM, RPD, TPM, and TPD, and those limits apply at the **organization** level rather than per user. <sup>[38]</sup>

Some useful currently listed model limits are:

- llama-3.1-8b-instant: 30 RPM, 14.4K RPD, 6K TPM, 500K TPD. <sup>[38]</sup>
- llama-3.3-70b-versatile: 30 RPM, 1K RPD, 12K TPM, 100K TPD. <sup>[38]</sup>
- meta-llama/llama-4-scout-17b-16e-instruct: 30 RPM, 1K RPD, 30K TPM, 500K TPD. <sup>[38]</sup>
- qwen/qwen3-32b: 60 RPM, 1K RPD, 6K TPM, 500K TPD. <sup>[38]</sup>

That means if your extraction pipeline depends on a stronger large model like llama-3.3-70b-versatile, your daily free usage is far lower than if you route simpler classification tasks to llama-3.1-8b-instant. <sup>[38]</sup>

## Crawl4AI cost reality

Crawl4AI itself is open-source and can be used without subscription fees, so there is no vendor-side per-request pricing like SerpAPI. <sup>[37]</sup>

But it is not “free” in an operational sense, because you still pay for:

- cloud VM or local machine time,
- browser automation overhead,
- memory/CPU for concurrent crawls,
- bandwidth,
- storage and logs,
- engineering time to keep crawlers stable. <sup>[40]</sup>

So your effective Crawl4AI cost is usually:

- **Local/student machine:** near \$0 direct spend, but slower and less reliable. <sup>[40]</sup>
- **Small VPS:** low monthly infra cost, often enough for modest daily crawls. <sup>[40]</sup>
- **Higher-scale production:** compute cost rises with concurrency, JavaScript-heavy pages, and retry volume. <sup>[40]</sup>

## Combined estimate

Here is a practical budget way to think about your setup:

| Scenario                      | Crawl4AI                  | SerpAPI                                           | Groq                                                                      | Estimated monthly spend                          |
|-------------------------------|---------------------------|---------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------|
| Prototype / hackathon         | Self-hosted, low infra    | Free tier 250 searches <sup>[39]</sup>            | Mostly free-tier limits <sup>[38]</sup>                                   | About \$0 to low infra-only cost <sup>[40]</sup> |
| Small production pilot        | Small VPS/self-host infra | Starter or Developer, \$25–\$75 <sup>[39]</sup>   | Often still free if prompts are selective <sup>[38]</sup>                 | Roughly infra + \$25 to \$75                     |
| Moderate daily lead engine    | Stronger infra            | Developer or Big Data, \$75–\$275 <sup>[39]</sup> | Free only if heavily optimized; otherwise monitor closely <sup>[38]</sup> | Roughly infra + \$75 to \$275                    |
| Heavy multi-source monitoring | Significant infra         | Big Data or enterprise <sup>[39]</sup>            | Likely needs paid Groq/developer upgrades if limits hit <sup>[38]</sup>   | Mostly driven by SerpAPI + infra + LLM scale     |

## Recommendation for your project

For your lead-identification system, the cheapest good architecture is:

- use **Crawl4AI** for crawling the actual public pages,
- use **SerpAPI** only for discovery such as targeted founder/company/job searches,
- use **Groq** selectively for unstructured extraction instead of on every page. <sup>[39]</sup> <sup>[40]</sup> <sup>[38]</sup>

That keeps SerpAPI calls low, preserves Groq's free-tier headroom, and uses Crawl4AI where it is strongest: browser-based page retrieval and structured extraction without per-call pricing.

[\[37\]](#) [\[39\]](#) [\[38\]](#)

A good initial planning assumption is:

- 250 free SerpAPI searches/month for testing, then \$25 or \$75 once the workflow is stable. [\[39\]](#)
- Groq free tier is enough if you reserve large-model calls for only the hard pages and lighter models for routine classification. [\[38\]](#)
- Crawl4AI has no API bill, but your machine or server becomes the bottleneck. [\[37\]](#) [\[40\]](#)

Would you like a sample cost model for 100, 1,000, and 10,000 leads/month based on your exact pipeline steps?

✱

## also mention that crawl and serp re in testing

Yes — you should explicitly state that Crawl4AI and SerpAPI are currently in testing / evaluation mode in your project, while Groq is part of the active enrichment/classification stack. Crawl4AI is open-source and self-hosted rather than a metered SaaS by default, and SerpAPI has a small free plan before paid tiers, so testing language is especially appropriate while you validate source coverage and search volume. [\[54\]](#) [\[55\]](#)

You can phrase it like this in your README or architecture notes:

### Suggested wording

- **Crawl4AI:** currently in testing as the primary crawling and extraction layer for public web pages; expected cost is near-zero on the software side because it is self-hosted/open-source, with only infrastructure cost for local or cloud execution. [\[55\]](#) [\[56\]](#)
- **SerpAPI:** currently in testing for search-driven discovery and lead sourcing; free plan includes 250 searches/month with 50 searches/hour, and paid plans begin at \$25/month if testing graduates to production usage. [\[54\]](#)
- **Groq:** used for selective LLM extraction/classification, with model-specific free-tier rate limits such as 1K RPD for llama-3.3-70b-versatile; production viability depends on how often LLM extraction is invoked. [\[57\]](#)

### Short version

You can also write it more compactly:

**Status:** Crawl4AI and SerpAPI are currently under testing/evaluation in this project. Crawl4AI is being assessed as a self-hosted crawl/extraction engine, while SerpAPI is being tested for discovery/search workflows. Groq remains the LLM layer for selective extraction and classification, subject to model-specific rate limits. [\[57\]](#) [\[55\]](#) [\[54\]](#)

## One-line note

If you want a one-line version for a tech stack table:

Crawl4AI and SerpAPI are in **testing**; Groq is used selectively for structured extraction/classification under current free-tier limits. [\[55\]](#) [\[54\]](#) [\[57\]](#)

If you want, I can now turn this into a **clean pricing + testing-status** section for your README.

✱

## also mention that crawl and serp are in testing phase and we're trying out different approaches

Yes — you can state it more clearly as an evaluation note:

**Status:** Crawl4AI and SerpAPI are currently in the **testing phase**, and we're trying different approaches to determine the most reliable and cost-effective setup for crawling, search-driven discovery, and lead enrichment. Crawl4AI is being evaluated as a self-hosted crawling layer with no mandatory API fees, while SerpAPI is being tested for search-based discovery under its free and paid usage tiers. Groq remains the LLM layer for selective extraction and classification, subject to organization-level rate limits. [\[69\]](#) [\[70\]](#) [\[71\]](#) [\[72\]](#)

## Shorter version

Crawl4AI and SerpAPI are currently in **testing**, and we're experimenting with different integration approaches before finalizing the production stack. Crawl4AI is attractive because it is self-hosted and free at the software level, while SerpAPI is being evaluated for discovery workflows with a free plan of 250 searches/month. Groq is used selectively for LLM-based extraction and classification. [\[70\]](#) [\[71\]](#) [\[69\]](#)

## README-ready version

If you want a polished README sentence, use this:

We are currently testing **Crawl4AI** and **SerpAPI** and trying different approaches for public-web crawling, search-driven lead discovery, and source coverage optimization. Crawl4AI is being explored as a self-hosted, no-mandatory-fee crawling option, while SerpAPI is under evaluation for query-based discovery. **Groq** is used selectively for extraction and classification, within its organization-level model rate limits. [\[72\]](#) [\[69\]](#) [\[70\]](#)

## One-line version

Crawl4AI and SerpAPI are in the **testing phase**, and we're actively trying different approaches before locking the final architecture; Groq is the current selective LLM layer. [\[69\]](#) [\[70\]](#) [\[72\]](#)

If you want, I can now combine this with the earlier cost details into one final polished paragraph/table for your README or PPT.

✱

## im getting this output in my terminal what does it indicate? is crawl4AI and serpAPI working?

From this terminal output: Crawl4AI is partially working, SerpAPI is not working, and Groq is also not working correctly in your current run. The log shows successful Crawl4AI fetches for several company pages, but SerpAPI repeatedly fails with an import error and Groq requests are returning HTTP 403 “Access denied.”<sup>[84]</sup>

### What each part indicates

#### Crawl4AI

Crawl4AI is **working** in some cases because your logs show repeated successful fetch/complete cycles such as:

- `FETCH ... COMPLETE for virohan.com, leadsquared.com, kekahr.com, getinge.com, jobgether.com, coinswitch.com, cwdlimited.com, and parts of ruskmedia.com.`<sup>[84]</sup>

But Crawl4AI is **not fully stable across all targets** because some sites are blocked or unreachable:

- `Wait condition failed Timeout after 20000ms waiting for selector networkidle on Antler/Surge/Wellfound pages,`<sup>[84]</sup>
- `netERRNAMENOTRESOLVED for some domains like lvlzerospotlight.com and doubletick.com,`<sup>[84]</sup>
- `navigation timeouts on nxtwave.com.`<sup>[84]</sup>

So the right interpretation is:

- Crawl4AI is installed and executing correctly.<sup>[84]</sup>
- It can crawl many pages successfully.<sup>[84]</sup>
- Some target sites are failing due to anti-bot protection, timeout/wait strategy, or invalid/unresolvable domains, not because Crawl4AI itself is broken.<sup>[84]</sup>

#### SerpAPI

SerpAPI is **not working** in this run.

The strongest signal is repeated warnings like:

- `SerpApi web search failed ... cannot import name GoogleSearch from serpapi`<sup>[84]</sup>

That means your code is failing **before making a valid SerpAPI search call**, most likely because:

- the installed `serpapi` package/version does not match the import your code expects, or
- you installed the wrong package/module, or
- your import path is outdated for the current SDK version. <sup>[84]</sup>

So no — based on this log, you cannot say SerpAPI is working. It is currently broken at the Python import layer. <sup>[84]</sup>

## Groq

Groq is also **not working** in this run.

Your logs repeatedly show:

- `POST https://api.groq.com/openai/v1/chat/completions HTTP/1.1 403 Forbidden`
- `LLM API error ... Access denied. Please check your network settings.` <sup>[84]</sup>

That means your code is reaching Groq’s endpoint, but the request is being denied. In practice this usually points to one of these:

- invalid API key,
- restricted network/IP/environment,
- wrong organization/project setup,
- wrong model/access permissions,
- malformed auth header or env loading issue. <sup>[84]</sup>

So the LLM-backed extraction/classification steps are failing in this run. <sup>[84]</sup>

## Overall status

Here’s the clean summary:

| Component | Status                   | Why                                                                                                                                  |
|-----------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Crawl4AI  | <b>Partially working</b> | Successful fetches/completions are visible, but some sites fail due to timeout, anti-bot, DNS, or navigation issues. <sup>[84]</sup> |
| SerpAPI   | <b>Not working</b>       | Repeated <code>cannot import name GoogleSearch from serpapi</code> errors show SDK/import failure. <sup>[84]</sup>                   |
| Groq      | <b>Not working</b>       | Repeated <code>403 Forbidden</code> and <code>Access denied</code> responses from Groq API. <sup>[84]</sup>                          |

## What your pipeline still achieved

Even with SerpAPI and Groq failing, your pipeline still ran a meaningful portion of the workflow:

- RSS scraper returned 10 candidates, <sup>[84]</sup>
- job scraper returned 15 candidates, <sup>[84]</sup>
- total raw candidates reached 25, later deduped to 23, then filtered to 19 valid ICP candidates, <sup>[84]</sup>

- scoring still ran and produced A1 B15 C3, <sup>[84]</sup>
- RAG dedup also ran successfully after loading the sentence-transformer model. <sup>[84]</sup>

So the system is **not fully broken**. It is degrading and continuing, which is good architecture behavior. But enrichment/search/LLM-dependent quality is currently reduced. <sup>[84]</sup>

## What the error patterns likely mean

### 1. `cannot import name GoogleSearch from serpapi`

This usually means your SerpAPI client code and installed package are mismatched. <sup>[84]</sup>

Common causes:

- wrong package installed,
- outdated/newer SDK mismatch,
- code written for one import style and environment has another.

### 2. `403 Forbidden` **from Groq**

This means the request reached Groq, but access was denied. <sup>[84]</sup>

Common causes:

- bad `GROQ_API_KEY`,
- key not loaded from `.env`,
- model not allowed,
- account/project/network restriction.

### 3. `Wait condition failed ... networkidle`

This often means your Crawl4AI config is waiting for a page state that never settles, especially on JS-heavy or ad-heavy pages. <sup>[84]</sup>

### 4. `netERRNAMENOTRESOLVED`

This means DNS/domain resolution failed for that target URL, so Crawl4AI could not reach the host at all. <sup>[84]</sup>

## Direct answer

So, to answer your question directly:

- **Is Crawl4AI working?**  
Yes, **partially**. It is definitely installed and crawling some pages successfully, but several targets are failing due to timeout, anti-bot, or DNS issues. <sup>[84]</sup>
- **Is SerpAPI working?**  
No. It is currently failing at the import stage, so those searches are not running. <sup>[84]</sup>

- Is Groq working?

No. Calls are being made, but Groq is returning 403 Access Denied. <sup>[84]</sup>

## What you should fix next

Fix in this order:

### 1. SerpAPI import issue

Because this is a straightforward code/package mismatch and easiest to unblock. <sup>[84]</sup>

### 2. Groq 403 issue

Check API key loading, env vars, and account/model access. <sup>[84]</sup>

### 3. Crawl4AI wait strategy

Replace `networkidle` for problematic sites with better per-site waits like:

- `domcontentloaded`,
- specific CSS selectors,
- shorter timeout + fallback path.

### 4. Bad/unresolvable domains

Add domain validation before crawl attempts.

## What I need from you to debug properly

Send these files and I can pinpoint the exact fixes:

- `utils/serp_client.py` or whatever SerpAPI wrapper you use
- `utils/llm_router.py`
- your `.env.example` keys section or redacted `.env` variable names
- the Crawl4AI wrapper/config file
- `requirements.txt`

The most important one first is your SerpAPI wrapper file, because the import error is explicit and likely easy to fix.

✱

# Google Search Engine Results API

[Release Notes](#)

[API uptime](#)

[99.980%](#)

Our Google Search API allows you to scrape results from the Google search page.

The API endpoint is <https://serpapi.com/search?engine=google>

Head to the [playground](#) for a live and interactive demo.

API Parameters

## Search Query

q

Required

Parameter defines the query you want to search. You can use anything that you would use in a regular Google search. e.g. inurl:, site:, intitle:. We also support advanced search query parameters such as as\_dt and as\_eq. See the [full list](#) of supported advanced search query parameters.

Geographic Location

location

Optional

Parameter defines from where you want the search to originate. If several locations match the location requested, we'll pick the most popular one. Head to the [/locations.json API](#) if you need more precise control. It is recommended to specify location at the city level in order to simulate a real user's search. If location is omitted, the search may take on the location of the proxy.

When only the location parameter is set, Google may still take into account the proxy's country, which can influence some results. For more consistent country-specific filtering, use the gl parameter alongside location.

Parameter can't be used together with uule, lat, and lon parameters.

uule

Optional

Parameter is the Google encoded location you want to use for the search.

uule and location parameters can't be used together.

Head to [UULE Converter](#) if you need to convert a location or GPS coordinates to uule parameter, and vice versa.

lat

Optional

Parameter defines a GPS latitude for the search origin.

Parameter is required when using lon parameter.

In some cases, matching gl parameter is required.

Parameter can't be used together with location and uule parameters.

lon

Optional

Parameter defines a GPS longitude for the search origin.

Parameter is required when using lat parameter.

In some cases, matching gl parameter is required.

Parameter can't be used together with location and uule parameters.

radius

Optional

Parameter defines the distance (in meters) within which to bias the search results. Results inside of this region will be ranked higher than results outside of it. However, prominent results from outside of the search radius may still be included.

Value range per device:

- Desktop: 1..199
- Tablet: 1..1000
- Mobile: 1..1000

Parameter can be used alongside location or lat/lon parameters.

When parameter is paired with location, its center point coordinates are used for the search.

Advanced Google Parameters

ludocid

Optional

Parameter defines the Google CID (customer identifier) of a place. This parameter can be found in [Google Search API](#) and [Google Local API](#) local results under the name of place\_id, as well as in [Google Maps API](#) local results under the name of data\_cid. You can also acquire it using [Google's CID converter](#).

lsig

Optional

Parameter that you might have to use to force the knowledge graph map view to show up. You can find the lsig ID by using our [Local Pack API](#) or [Google Local API](#).

lsig ID is also available via a redirect Google uses within [Google My Business](#).

kgmid

Optional

Parameter defines the id (KG MID) of the Google Knowledge Graph listing you want to scrape. Also known as Google Knowledge Graph ID. Searches with kgmid parameter will return results for the originally encrypted search parameters. For some searches, kgmid may override all other parameters except start parameter.

si

Optional

Parameter defines the cached search parameters of the Google Search you want to scrape. Searches with si parameter will return results for the originally encrypted search parameters. For some searches, si may override all other parameters except startsi can be used to scrape Google Knowledge Graph Tabs.

ibp

Optional

Parameter is responsible for rendering layouts and expansions for some elements (e.g., gwp;0,7 to expand searches with ludocid for expanded knowledge graph).

uds

Optional

Parameter enables to filter search. It's a string provided by Google as a filter. uds values are

provided under the section: filters with uds, q and serpapi\_link values provided for each filter.  
Localization

google\_domain

Optional

Parameter defines the Google domain to use. It defaults to [google.com](https://www.google.com). Head to the [Google domains page](#) for a full list of supported Google domains.

gl

Optional

Parameter defines the country to use for the Google search. It's a two-letter country code. (e.g., us for the United States, uk for United Kingdom, or fr for France). Head to the [Google countries page](#) for a full list of supported Google countries.

hl

Optional

Parameter defines the language to use for the Google search. It's a two-letter language code. (e.g., en for English, es for Spanish, or fr for French). Head to the [Google languages page](#) for a full list of supported Google languages.

cr

Optional

Parameter defines one or multiple countries to limit the search to. It uses country{two-letter upper-case country code} to specify countries and | as a delimiter. (e.g., countryFR|countryDE will only search French and German pages). Head to the [Google cr countries page](#) for a full list of supported countries.

lr

Optional

Parameter defines one or multiple languages to limit the search to. It uses lang\_{two-letter language code} to specify languages and | as a delimiter. (e.g., lang\_fr|lang\_de will only search French and German pages). Head to the [Google lr languages page](#) for a full list of supported languages.

Advanced Filters

tbs

Optional

(to be searched) parameter defines advanced search parameters that aren't possible in the regular query field. (e.g., advanced search for patents, dates, news, videos, images, apps, or text contents).

safe

Optional

Parameter defines the level of filtering for adult content. It can be set to active or off, by default Google will blur explicit content.

nfpr

Optional

Parameter defines the exclusion of results from an auto-corrected query when the original

query is spelled wrong. It can be set to 1 to exclude these results, or 0 to include them (default). Note that this parameter may not prevent Google from returning results for an auto-corrected query if no other results are available.

filter

Optional

Parameter defines if the filters for 'Similar Results' and 'Omitted Results' are on or off. It can be set to 1 (default) to enable these filters, or 0 to disable these filters.

Search Type

tbm

Optional

(to be matched) parameter defines the type of search you want to do.

It can be set to:

(no tbm parameter): regular Google Search,

isch: [Google Images API](#),

lcl: [Google Local API](#),

vid: [Google Videos API](#),

nws: [Google News API](#),

shop: [Google Shopping API](#),

pts: [Google Patents API](#),

or any other Google service.

Pagination

start

Optional

Parameter defines the result offset. It skips the given number of results. It's used for pagination. (e.g., 0 (default) is the first page of results, 10 is the 2nd page of results, 20 is the 3rd page of results, etc.).

Serpapi Parameters

engine

Optional

Set parameter to google (default) to use the Google API engine.

device

Optional

Parameter defines the device to use to get the results. It can be set to desktop (default) to use a regular browser, tablet to use a tablet browser (currently using iPads), or mobile to use a mobile browser.

no\_cache

Optional

Parameter will force SerpApi to fetch the Google results even if a cached version is already present. A cache is served only if the query and all parameters are exactly the same. Cache expires after 1h. Cached searches are free, and are not counted towards your searches per

month. It can be set to false (default) to allow results from the cache, or true to disallow results from the cache. `no_cache` and `async` parameters should not be used together.

`async`

Optional

Parameter defines the way you want to submit your search to SerpApi. It can be set to false (default) to open an HTTP connection and keep it open until you got your search results, or true to just submit your search to SerpApi and retrieve them later. In this case, you'll need to use our [Searches Archive API](#) to retrieve your results. `async` and `no_cache` parameters should not be used together. `async` should not be used on accounts with [Ludicrous Speed](#) enabled.

`zero_trace`

Optional

Enterprise only. Parameter enables [ZeroTrace mode](#). It can be set to false (default) or true. Enable this mode to skip storing search parameters, search files, and search metadata on our servers. This may make debugging more difficult.

`api_key`

Required

Parameter defines the SerpApi private key to use.

`output`

Optional

Parameter defines the final output you want. It can be set to `json` (default) to get a structured JSON of the results, or `html` to get the raw html retrieved.

API Results

JSON Results

JSON output includes structured data for [organic results](#), [local results](#), [ad results](#), the [knowledge graph](#), [direct answer boxes](#), [images results](#), [news results](#), [shopping results](#), [video results](#), and more.

A search status is accessible through `search_metadata.status`. It flows this way: Processing -

> Success || Error. If a search has failed, error will contain an error

message. `search_metadata.id` is the search ID inside SerpApi.

HTML Results

HTML output is useful to debug JSON results or support features not supported yet by SerpApi.

HTML output gives you the raw HTML results from Google.

this is serp ai documentation for google search

this is our current serp code

```
""SerpApi client — Google Search/News discovery.
```

All search functions fall back to DuckDuckGo HTML scraping if `SERPAPI_KEY` is not configured, so the system runs zero-cost without a key.

Usage:

```
from utils.serp_client import google_news_search, google_web_search
```

```

# Funding verification
articles = await google_news_search("Coram AI funding 2025")
# -> [{title: ..., "link": ..., "snippet": ..., "date": ...}]

# LinkedIn/website discovery
results = await google_web_search('site:linkedin.com/in "John Doe" "Coram AI"')
"""

import asyncio
import logging
import re
from urllib.parse import quote_plus

logger = logging.getLogger(name)

def _get_key() -> str:
    from config.settings import SERPAPI_KEY
    return SERPAPI_KEY or ""

```

## — Public API

---



---

```

async def google_news_search(
    query: str,
    lookback_months: int = 2,
    num: int = 10,
) -> list[dict]:
    """Search Google News. Returns [{title, link, snippet, date}].

    Uses SerpApi when key configured, else DuckDuckGo fallback.
    """
    key = _get_key()
    if key:
        results = await _serpapi_news(query, lookback_months, key, num)
        if results:
            return results
    return await _ddg_search(query, news=True)

async def google_web_search(
    query: str,
    num: int = 5,
) -> list[dict]:
    """Search Google web. Returns [{title, link, snippet}].

    Uses SerpApi when key configured, else DuckDuckGo fallback.
    """
    key = _get_key()

```

if key:

    results = await \_serpapi\_web(query, num, key)

if results:

    return results

return await \_ddg\_search(query, news=False)

async def find\_linkedin\_profile(founder\_name: str, company: str) -> str:

    """Return LinkedIn profile URL for a founder at a company, or empty string."""

    query = f'site:linkedin.com/in "{founder\_name}" "{company}"'

    results = await google\_web\_search(query, num=3)

    for r in results:

        link = r.get("link", "")

        if "linkedin.com/in/" in link:

            return link

    # Try snippet links

    for r in results:

        m = re.search(r"https?:\/\/[a-z]{0,3}?.?linkedin.com/in/[A-Za-z0-9-\_%]\+", r.get("snippet", ""))

        if m:

            return m.group(0)

    return ""

async def find\_company\_website(company: str) -> str:

    """Return the most likely official website domain for a company, or empty."""

    query = f'"{company}" India startup official website'

    results = await google\_web\_search(query, num=3)

\_skip\_domains = {

    "linkedin.com", "wellfound.com", "crunchbase.com", "yourstory.com",

    "inc42.com", "entrackr.com", "vccircle.com", "economictimes.indiatimes.com",

    "livemint.com", "moneycontrol.com", "business-standard.com",

    "tracxn.com", "tofler.in", "zaubacorp.com", "ycombinator.com",

    "twitter.com", "facebook.com", "instagram.com", "wikipedia.org",

}

for r in results:

    link = r.get("link", "")

    if not link:

        continue

    try:

        from urllib.parse import urlparse

        host = urlparse(link).netloc.lower().lstrip("www.")

        if not any(skip in host for skip in \_skip\_domains):

            return host

    except Exception:

        continue

return ""

```
async def verify_company_exists(company: str) -> dict:
```

```
    """Check if a company exists via Google web/news search.
```

```
Returns {"exists": bool, "confidence": str, "sources": int, "reason": str}
```

```
    """
```

```
    query = f"""{company}" India startup company'
```

```
    results = await google_web_search(query, num=5)
```

```
    credible_sources = {
```

```
        "linkedin.com", "yourstory.com", "inc42.com", "entrackr.com",
```

```
        "vccircle.com", "economictimes.indiatimes.com", "livemint.com",
```

```
        "crunchbase.com", "ycombinator.com", "tracxn.com", "wellfound.com",
```

```
        "techcrunch.com", "business-standard.com", "moneycontrol.com",
```

```
        "startupindia.gov.in", "tofler.in", "zaubacorp.com",
```

```
    }
```

```
    found_on: list[str] = []
```

```
    for r in results:
```

```
        link = r.get("link", "")
```

```
        snippet = r.get("snippet", "")
```

```
        title = r.get("title", "")
```

```
        if company.lower() in (snippet + title).lower():
```

```
            try:
```

```
                from urllib.parse import urlparse
```

```
                host = urlparse(link).netloc.lower().lstrip("www.")
```

```
                if any(src in host for src in credible_sources):
```

```
                    found_on.append(host)
```

```
            elif host:
```

```
                found_on.append(host)
```

```
        except Exception:
```

```
            pass
```

```
n = len(found_on)
```

```
    if n >= 3:
```

```
        return {"exists": True, "confidence": "high", "sources": n, "reason": f"found on {',
```

```
        '.join(found_on[:3])}'"}"
```

```
    if n >= 1:
```

```
        return {"exists": True, "confidence": "medium", "sources": n, "reason": f"found on
```

```
        {found_on[0]}"}"
```

```
    return {"exists": False, "confidence": "low", "sources": 0, "reason": "not found in web search"}
```

# — SerpApi implementations

---

```
async def _serpapi_news(query: str, lookback_months: int, api_key: str, num: int) -> list[dict]:
    try:
        from serpapi import GoogleSearch
        params = {
            "engine": "google",    # ✓ NOT "google_news"
            "tbm": "nws",         # ✓ switches to news results
            "q": query,
            "gl": "in",
            "hl": "en",
            "tbs": f"qdr:m{lookback_months}",
            "num": num,
            "api_key": api_key,
        }

    def _run():
        return GoogleSearch(params).get_dict()

    data = await asyncio.get_event_loop().run_in_executor(None, _run)

    # news via tbm=nws returns organic_results, not news_results
    news = data.get("organic_results") or data.get("news_results") or []

    return [
        {
            "title": r.get("title", ""),
            "link": r.get("link", ""),
            "snippet": r.get("snippet", ""),
            "date": r.get("date", ""),
        }
        for r in news[:num]
    ]

    except Exception as exc:
        logger.warning("SerpApi news search failed ('%s'): %s", query[:60], exc)
        return []

async def _serpapi_web(query: str, num: int, api_key: str) -> list[dict]:
    try:
        from serpapi import GoogleSearch
        params = {
            "engine": "google", # ✓ correct, no change needed here
            "q": query,
            "gl": "in",
            "hl": "en",
```

```

        "num": num,
        "api_key": api_key,
    }

def _run():
    return GoogleSearch(params).get_dict()

data = await asyncio.get_event_loop().run_in_executor(None, _run)
organic = data.get("organic_results", [])
return [
    {
        "title": r.get("title", ""),
        "link": r.get("link", ""),
        "snippet": r.get("snippet", ""),
    }
    for r in organic[:num]
]
except Exception as exc:
    logger.warning("SerpApi web search failed ('%s'): %s", query[:60], exc)
    return []

```

## — DuckDuckGo HTML fallback

---

```

async def _ddg_search(query: str, news: bool = False) -> list[dict]:
    """DuckDuckGo HTML search — zero-cost fallback when no SerpApi key."""
    from utils.http_fetch import fetch_as_text
    q = quote_plus(query)
    url = f"https://html.duckduckgo.com/html/?q={q}"
    if news:
        url += "&iar=news"

    content = await fetch_as_text(url, max_chars=6000)
    if not content:
        return []

    # Parse result blocks from DDG HTML
    results: list[dict] = []
    # Split on result separators
    blocks = re.split(r"(?=result__title|result__url|result__snippet)", content)

    title_re = re.compile(r"result__a[^\>]>(.*?)</", re.DOTALL)
    snippet_re = re.compile(r"result__snippet[^\>]>(.*?)</", re.DOTALL)
    url_re = re.compile(r"result__url[^\>]>(.*?)</", re.DOTALL)

```

```

for block in blocks[:8]:
    title_m = title_re.search(block)
    snippet_m = snippet_re.search(block)
    url_m = url_re.search(block)
    if not (title_m or snippet_m):
        continue
    results.append({
        "title": re.sub(r"<[^>]+>", "", title_m.group(1)).strip() if title_m else "",
        "link": re.sub(r"<[^>]+>", "", url_m.group(1)).strip() if url_m else "",
        "snippet": re.sub(r"<[^>]+>", "", snippet_m.group(1)).strip() if snippet_m else "",
        "date": "",
    })

return results[:6]

```

give me the corrected code

Yes — the main issue in your current code is the **SerpAPI Python import path/package mismatch**. SerpApi’s Python integration uses the `google-search-results` package, and examples commonly import `GoogleSearch` from `serpapi.google_search`, which avoids the exact `cannot import name GoogleSearch from serpapi` problem you saw in your logs. [\[85\]](#) [\[86\]](#) [\[87\]](#)

Below is a **corrected version** of your `utils/serp_client.py` that:

- uses the safer import path,
- keeps your DuckDuckGo fallback,
- adds basic `search_metadata.status` handling,
- uses `tbm="nws"` correctly for Google News,
- returns clean normalized results,
- logs failures more clearly. [\[86\]](#) [\[88\]](#)

## Corrected code

```

"""SerpApi client – Google Search/News discovery.

All search functions fall back to DuckDuckGo HTML scraping if SERPAPI_KEY
is not configured or SerpApi fails, so the system can still run with no key.

Requires:
    pip install google-search-results

Usage:
    from utils.serp_client import google_news_search, google_web_search

    articles = await google_news_search("Coram AI funding 2025")
    results = await google_web_search('site:linkedin.com/in "John Doe" "Coram AI"')
"""

import asyncio

```

```

import logging
import re
from urllib.parse import quote_plus, urlparse

logger = logging.getLogger(__name__)

def _get_key() -> str:
    from config.settings import SERPAPI_KEY
    return (SERPAPI_KEY or "").strip()

def _import_google_search():
    try:
        from serpapi.google_search import GoogleSearch
        return GoogleSearch
    except Exception:
        try:
            from serpapi import GoogleSearch
            return GoogleSearch
        except Exception as exc:
            raise ImportError(
                "Could not import GoogleSearch from SerpApi. "
                "Install the correct package with: pip install google-search-results"
            ) from exc

# — Public API —————

async def google_news_search(
    query: str,
    lookback_months: int = 2,
    num: int = 10,
) -> list[dict]:
    """Search Google News. Returns [{title, link, snippet, date}]."""
    key = _get_key()
    if key:
        results = await _serpapi_news(query, lookback_months, key, num)
        if results:
            return results
    return await _ddg_search(query, news=True)

async def google_web_search(
    query: str,
    num: int = 5,
) -> list[dict]:
    """Search Google web. Returns [{title, link, snippet}]."""
    key = _get_key()
    if key:
        results = await _serpapi_web(query, num, key)
        if results:
            return results
    return await _ddg_search(query, news=False)

```

```

async def find_linkedin_profile(founder_name: str, company: str) -> str:
    """Return LinkedIn profile URL for a founder at a company, or empty string."""
    query = f'site:linkedin.com/in "{founder_name}" "{company}'
    results = await google_web_search(query, num=3)

    for r in results:
        link = r.get("link", "")
        if "linkedin.com/in/" in link:
            return link

    for r in results:
        m = re.search(
            r"https://(?:[a-z]{0,3}\.)?linkedin\.com/in/[A-Za-z0-9\-\_%]+",
            r.get("snippet", "")
        )
        if m:
            return m.group(0)

    return ""

async def find_company_website(company: str) -> str:
    """Return the most likely official website domain for a company, or empty."""
    query = f'"{company}" India startup official website'
    results = await google_web_search(query, num=5)

    skip_domains = {
        "linkedin.com", "wellfound.com", "crunchbase.com", "yourstory.com",
        "inc42.com", "entrackr.com", "vccircle.com", "economictimes.indiatimes.com",
        "livemint.com", "moneycontrol.com", "business-standard.com",
        "tracxn.com", "tofler.in", "zaubacorp.com", "ycombinator.com",
        "twitter.com", "x.com", "facebook.com", "instagram.com", "wikipedia.org",
    }

    for r in results:
        link = r.get("link", "")
        if not link:
            continue
        try:
            host = urlparse(link).netloc.lower().lstrip("www.")
            if host and not any(skip in host for skip in skip_domains):
                return host
        except Exception:
            continue

    return ""

async def verify_company_exists(company: str) -> dict:
    """Check if a company exists via Google web search.

    Returns:
        {"exists": bool, "confidence": str, "sources": int, "reason": str}
    """
    query = f'"{company}" India startup company'
    results = await google_web_search(query, num=5)

```

```

credible_sources = {
    "linkedin.com", "yourstory.com", "inc42.com", "entrackr.com",
    "vccircle.com", "economictimes.indiatimes.com", "livemint.com",
    "crunchbase.com", "ycombinator.com", "tracxn.com", "wellfound.com",
    "techcrunch.com", "business-standard.com", "moneycontrol.com",
    "startupindia.gov.in", "tofler.in", "zaubacorp.com",
}

found_on: list[str] = []

for r in results:
    link = r.get("link", "")
    snippet = r.get("snippet", "")
    title = r.get("title", "")

    if company.lower() in (title + " " + snippet).lower():
        try:
            host = urlparse(link).netloc.lower().lstrip("www.")
            if not host:
                continue
            found_on.append(host)
        except Exception:
            continue

found_on = list(dict.fromkeys(found_on))
credible_hits = [h for h in found_on if any(src in h for src in credible_sources)]

if len(credible_hits) >= 2:
    return {
        "exists": True,
        "confidence": "high",
        "sources": len(credible_hits),
        "reason": f"found on {' '.join(credible_hits[:3])}",
    }
if len(found_on) >= 1:
    return {
        "exists": True,
        "confidence": "medium",
        "sources": len(found_on),
        "reason": f"found on {found_on[0]}",
    }
return {
    "exists": False,
    "confidence": "low",
    "sources": 0,
    "reason": "not found in web search",
}

```

# — SerpApi implementations —————

```

async def _serpapi_news(query: str, lookback_months: int, api_key: str, num: int) ->
    try:
        GoogleSearch = _import_google_search()

```

```

params = {
    "engine": "google",
    "tbs": "nws",
    "q": query,
    "gl": "in",
    "hl": "en",
    "google_domain": "google.co.in",
    "tbs": f"qdr:m{lookback_months}",
    "num": num,
    "api_key": api_key,
    "output": "json",
}

def _run():
    return GoogleSearch(params).get_dict()

data = await asyncio.get_event_loop().run_in_executor(None, _run)

status = (((data or {}).get("search_metadata") or {}).get("status") or "").lower()
if status == "error" or data.get("error"):
    logger.warning(
        "SerpApi news error ('%s'): %s",
        query[:80],
        data.get("error") or data.get("search_metadata", {}).get("status"),
    )
    return []

news = data.get("news_results") or data.get("organic_results") or []

return [
    {
        "title": r.get("title", "") or "",
        "link": r.get("link", "") or "",
        "snippet": r.get("snippet", "") or "",
        "date": r.get("date", "") or "",
    }
    for r in news[:num]
    if r.get("title") or r.get("link")
]

except Exception as exc:
    logger.warning("SerpApi news search failed ('%s'): %s", query[:80], exc)
    return []

async def _serpapi_web(query: str, num: int, api_key: str) -> list[dict]:
    try:
        GoogleSearch = _import_google_search()

        params = {
            "engine": "google",
            "q": query,
            "gl": "in",
            "hl": "en",
            "google_domain": "google.co.in",
            "num": num,

```

```

        "api_key": api_key,
        "output": "json",
    }

    def _run():
        return GoogleSearch(params).get_dict()

    data = await asyncio.get_event_loop().run_in_executor(None, _run)

    status = (((data or {}).get("search_metadata") or {}).get("status") or "").lower()
    if status == "error" or data.get("error"):
        logger.warning(
            "SerpApi web error ('%s'): %s",
            query[:80],
            data.get("error") or data.get("search_metadata", {}).get("status"),
        )
        return []

    organic = data.get("organic_results", [])

    return [
        {
            "title": r.get("title", "") or "",
            "link": r.get("link", "") or "",
            "snippet": r.get("snippet", "") or "",
            "date": r.get("date", "") or "",
        }
        for r in organic[:num]
        if r.get("title") or r.get("link")
    ]

except Exception as exc:
    logger.warning("SerpApi web search failed ('%s'): %s", query[:80], exc)
    return []

# — DuckDuckGo HTML fallback —————

async def _ddg_search(query: str, news: bool = False) -> list[dict]:
    """DuckDuckGo HTML search – zero-cost fallback when no SerpApi key."""
    from utils.http_fetch import fetch_as_text

    q = quote_plus(query)
    url = f"https://html.duckduckgo.com/html/?q={q}"
    if news:
        url += "&iar=news"

    content = await fetch_as_text(url, max_chars=12000)
    if not content:
        return []

    results: list[dict] = []

    result_links = re.findall(
        r'<a[^\>]+class="[^"]*"result_a["^"]*">+href="([^\"]+)">(.*)</a>',
        content,
    )

```

```

        re.DOTALL,
    )
    snippets = re.findall(
        r'<a[^>]+class="[^"]*result__snippet["^"]*">(.*)</a>|<div[^>]+class="[^"]*result__snippet["^"]*">(.*)</div>',
        content,
        re.DOTALL,
    )

    for i, (href, title_html) in enumerate(result_links[:8]):
        snippet_html = ""
        if i < len(snippets):
            snippet_html = snippets[i][^8_0] or snippets[i][^8_1] or ""

        title = re.sub(r"<[>]+>", "", title_html).strip()
        link = re.sub(r"<[>]+>", "", href).strip()
        snippet = re.sub(r"<[>]+>", "", snippet_html).strip()

        results.append({
            "title": title,
            "link": link,
            "snippet": snippet,
            "date": "",
        })

    return results[:6]

```

## What else to change

Update your `requirements.txt` to include the official SerpApi Python package:

```
google-search-results
```

That package is the one referenced by SerpApi's Python integration examples. [\[89\]](#) [\[86\]](#)

## Important note

Your previous terminal log showed:

- `cannot import name GoogleSearch from serpapi, which this code is meant to fix by trying from serpapi.google_search import GoogleSearch first`, [\[90\]](#) [\[85\]](#)
- but if the wrong package is installed, you still need to reinstall the dependency properly with `pip install google-search-results`. [\[91\]](#) [\[85\]](#)

## Quick test

Use this after updating:

```

results = await google_web_search('site:linkedin.com/in "John Doe" "OpenAI"', num=3)
print(results[:2])

```

If SerpApi still fails, your code should now fall back cleanly to DuckDuckGo instead of crashing.  
[90]

## One more thing

Your Groq errors are separate from SerpAPI. The corrected Serp code will not fix the 403 Forbidden Groq issue shown in your logs. [90]

If you want, next I can give you:

1. a **fully cleaned production version** of this file with retries/timeouts, or
2. the exact **requirements + test script** to validate SerpAPI independently before plugging it back into your pipeline.

✱✱

## this is our folder structure

this is our llm\_router.py

```
"""LLM Router — round-robin across multiple API keys (OpenAI-compatible).
```

Backed by OpenRouter (<https://openrouter.ai>) using the openai SDK.

Model: set LLM\_MODEL in .env (default: openai/gpt-oss-120b).

Base URL: set LLM\_BASE\_URL in .env (default: <https://openrouter.ai/api/v1>).

When one key hits a rate limit (HTTP 429 / RateLimitError), the router automatically switches to the next key. Cycles indefinitely until all keys are exhausted in the current window, then waits and retries.

Usage:

```
from utils.llm_router import router
response = await router.chat(messages=[...], max_tokens=100)
"""
```

```
import asyncio
```

```
import logging
```

```
import time
```

```
from dataclasses import dataclass
```

```
from typing import Optional
```

```
from config.settings import LLM_API_KEYS, LLM_MODEL, LLM_BASE_URL
```

# Keep legacy names so any code importing GROQ\_\* still resolves

```
GROQ_API_KEYS = LLM_API_KEYS
```

```
GROQ_MODEL = LLM_MODEL
```

```
logger = logging.getLogger(name)
```

```
@dataclass
```

```
class KeySlot:
```

```
    api_key: str
```

```
    calls_this_window: int = 0
```

```
    window_reset_at: float = 0.0    # epoch seconds when window resets
```

```
    rate_limited_until: float = 0.0  # epoch seconds — back-off until this time
```

```
    total_calls: int = 0
```

```
    total_failures: int = 0
```

```
def is_available(self) -> bool:
```

```
    return time.time() >= self.rate_limited_until
```

```
def mark_rate_limited(self, retry_after_seconds: float = 60.0) -> None:
```

```
    self.rate_limited_until = time.time() + retry_after_seconds
```

```
    self.total_failures += 1
```

```
    logger.warning(
```

```
        "Key ...%s rate-limited. Back-off for %.0fs",
```

```
        self.api_key[-6:], retry_after_seconds
```

```
    )
```

```
def record_success(self) -> None:
```

```
    self.calls_this_window += 1
```

```
    self.total_calls += 1
```

```
class GroqRouter:
```

```
    """
```

```
    Thread-safe round-robin LLM router (OpenAI-compatible via OpenRouter).
```

```
    Class name kept as GroqRouter so all existing imports continue to work.
```

```
    """
```

```
def init(self, api_keys: list[str], model: str = LLM_MODEL) -> None:
```

```
    if not api_keys:
```

```
        raise ValueError("At least one API key is required")
```

```
    self.slots = [KeySlot(key) for key in api_keys]
```

```
    self.model = model
```

```
    self.base_url = LLM_BASE_URL
```

```
    self._index = 0
```

```
    self._lock = asyncio.Lock()
```

```
    logger.info(
```

```
        "LLM Router initialized: model=%s base_url=%s keys=%d",
```

```

        self.model, self.base_url, len(self.slots),
    )

def _next_available(self) -> Optional[KeySlot]:
    """Return next available slot in round-robin order. None if all rate-limited."""
    n = len(self.slots)
    for i in range(n):
        slot = self.slots[(self._index + i) % n]
        if slot.is_available():
            self._index = (self._index + i + 1) % n
            return slot
    return None

async def chat(
    self,
    messages: list[dict],
    max_tokens: int = 256,
    temperature: float = 0.0,
    retries: int = 3,
) -> str:
    """
    Send a chat completion request, rotating keys on rate limit.
    Returns the response text string.
    Raises RuntimeError if all keys are exhausted after retries attempts.
    """

    from openai import OpenAI, RateLimitError, APIStatusError

    for attempt in range(retries):
        async with self._lock:
            slot = self._next_available()

        if slot is None:
            # All keys rate-limited — find the soonest recovery time
            soonest = min(s.rate_limited_until for s in self.slots)
            wait = max(0.0, soonest - time.time()) + 1.0
            logger.warning("All LLM keys rate-limited. Waiting %.1fs...", wait)
            await asyncio.sleep(wait)
            async with self._lock:
                slot = self._next_available()
            if slot is None:
                raise RuntimeError("All LLM API keys are rate-limited")

    try:
        # openai SDK is synchronous — run in thread pool to stay async-safe
        def _call():
            client = OpenAI(api_key=slot.api_key, base_url=self.base_url)
            return client.chat.completions.create(
                model=self.model,

```

```

        messages=messages,
        max_tokens=max_tokens,
        temperature=temperature,
    )

response = await asyncio.get_event_loop().run_in_executor(None, _call)
    slot.record_success()
    return response.choices[0].message.content.strip()

except RateLimitError as exc:
    retry_after = 60.0
    try:
        retry_after = float(exc.response.headers.get("retry-after", 60))
    except Exception:
        pass
    slot.mark_rate_limited(retry_after)
    logger.info("Switching to next key after rate limit (attempt %d/%d)", attempt + 1, retries)

except APIStatusError as exc:
    if exc.status_code == 429:
        slot.mark_rate_limited(60.0)
    else:
        logger.error("LLM API error: %s", exc)
        raise

except Exception as exc:
    logger.error("LLM unexpected error (key ...%s): %s", slot.api_key[-6:], exc)
    raise

raise RuntimeError(f"LLM chat failed after {retries} attempts across all keys")

def status(self) -> list[dict]:
    """Return per-key status for logging/monitoring."""
    now = time.time()
    return [
        {
            "key_suffix": s.api_key[-6:],
            "available": s.is_available(),
            "total_calls": s.total_calls,
            "total_failures": s.total_failures,
            "limited_for_sec": max(0.0, round(s.rate_limited_until - now, 1)),
        }
        for s in self.slots
    ]

```

# Singleton — import this in all agents

```
def _build_router() -> Optional[GroqRouter]:
    if not LLM_API_KEYS:
        logger.warning("No LLM API keys configured — LLM features disabled")
        return None
    return GroqRouter(api_keys=LLM_API_KEYS, model=LLM_MODEL)

router: Optional[GroqRouter] = _build_router()
```

so wehre do you think the groq issue is

The most likely Groq issue is **not the router logic itself**. Your router is structurally fine for round-robin retries, but your logs show **HTTP 403 Forbidden / Access denied**, which points much more strongly to a **configuration or permissions problem**: wrong base URL, wrong key loaded, wrong model name for that provider/project, or provider mismatch between Groq and OpenRouter. Groq documents that 403 means the request is not allowed due to permission restrictions, and model-permission restrictions can also return 403 for all keys in an organization or project. <sup>[101]</sup> <sup>[102]</sup> <sup>[103]</sup>

## Main problem

Your `llm_router.py` says it is “Backed by OpenRouter” and defaults to `LLM_BASE_URL`, but your terminal log shows requests going to `https://api.groq.com/openai/v1/chat/completions`, not OpenRouter. <sup>[103]</sup>

That means one of these is happening:

- `LLM_BASE_URL` is set to Groq, not OpenRouter. <sup>[103]</sup>
- `LLM_MODEL` may still be an OpenRouter-style model string while the endpoint is Groq. <sup>[103]</sup>
- your keys in `LLM_API_KEYS` may be OpenRouter keys, but the code is calling Groq’s endpoint. <sup>[103]</sup>
- or you intended Groq directly, but the model or project permissions do not allow that model. Groq says restricted models return 403 `permissions_error`. <sup>[102]</sup>

So the first thing I would suspect is a **provider mismatch**:

- OpenRouter key + Groq base URL,
- or Groq key + OpenRouter model name,
- or Groq base URL + model not allowed in that Groq project. <sup>[104]</sup> <sup>[102]</sup>

## Why I don’t think the core router is the issue

Your router:

- rotates keys,
- catches `RateLimitError`,

- catches `APIStatusError`,
- uses the OpenAI-compatible client correctly in principle,
- and sends requests through `client.chat.completions.create(...)`.

That part is reasonable. A bad router usually causes crashes, missing retries, or wrong concurrency behavior. Your logs instead show the requests successfully reaching the API endpoint and being rejected with 403, which is an auth/permissions/configuration signal, not a round-robin bug.<sup>[101]</sup> <sup>[103]</sup>

## Most likely causes

### 1. Base URL and key mismatch

If you use Groq directly, the base URL should be Groq's OpenAI-compatible endpoint and the key must be a Groq key. Guidance for Groq OpenAI-compatible usage is to initialize the OpenAI client with `base_url="https://api.groq.com/openai/v1"` and a valid `GROQ_API_KEY`; using the wrong key or wrong base URL causes auth failures.<sup>[104]</sup>

### 2. Model not allowed for the current provider/project

Groq explicitly says restricted model access returns 403, and permissions are applied at the organization/project level, not per API key.<sup>[102]</sup>

So if your `.env` has something like an OpenRouter model name such as:

- `openai/gpt-oss-120b`  
or any non-Groq model ID,  
and you send that to Groq's endpoint, it can fail with 403 or equivalent permissions/access errors.<sup>[105]</sup> <sup>[102]</sup>

### 3. Wrong env wiring

You renamed everything to `LLM_*`, but kept `GROQ_*` aliases in code:

```
GROQ_API_KEYS = LLM_API_KEYS
GROQ_MODEL = LLM_MODEL
```

That is okay internally, but it increases the chance that:

- `.env` still has `GROQ_API_KEY`,
- `config.settings` now expects `LLM_API_KEYS`,
- and the wrong variable is actually being loaded.

If the wrong keys are loaded, every call can produce 403.<sup>[103]</sup>

## 4. Provider abstraction is mixed

Your comment says:

- “Backed by OpenRouter”
- “Model default: openai/gpt-oss-120b”
- “Base URL default: <https://openrouter.ai/api/v1>”

But your runtime log shows Groq endpoint usage. That inconsistency is the biggest red flag in the whole setup. <sup>[103]</sup>

### What I would check immediately

Check these four values at runtime:

- LLM\_BASE\_URL
- LLM\_MODEL
- number of LLM\_API\_KEYS
- first/last few chars of one loaded key

If:

- LLM\_BASE\_URL = `https://api.groq.com/openai/v1`
- but LLM\_MODEL = `openai/gpt-oss-120b`  
then that is almost certainly wrong. <sup>[102]</sup> <sup>[103]</sup>

If:

- LLM\_BASE\_URL = `https://openrouter.ai/api/v1`
- but logs still hit Groq,  
then some other config path is overriding the endpoint. <sup>[103]</sup>

### Code-level weak spots

These are not necessarily the 403 cause, but they are worth fixing:

`max_tokens`

OpenRouter documents `max_completion_tokens` as the preferred parameter, with `max_tokens` deprecated but still supported. This is usually not a 403 cause, but for better compatibility I'd modernize it. <sup>[106]</sup> <sup>[107]</sup>

### No provider validation

Right now the router accepts any base URL and any model string. That makes provider mismatch easy. You should validate:

- if base URL contains `groq`, model must be from allowed Groq models,
- if base URL contains `openrouter`, use an OpenRouter-style model ID. <sup>[104]</sup> <sup>[102]</sup>

## No startup config log safety check

You log model/base URL/key count, which is good, but you should also warn if:

- base URL is Groq and model looks like OpenRouter,
- base URL is OpenRouter and key prefix looks like Groq,
- base URL is Groq and no GROQ-compatible model naming is used.

## My diagnosis

The Groq issue is most likely in configuration, not in the retry logic:

1. wrong provider endpoint vs key/model combination is my top suspect, [\[104\]](#) [\[103\]](#)
2. model permission restriction on Groq is second, [\[102\]](#)
3. env vars not loading the expected keys is third. [\[103\]](#)

## What to do next

Add one startup debug log in `llm_router.py` to confirm the actual runtime config:

```
logger.info(
    "LLM runtime check: base_url=%s model=%s keys=%d first_key_suffix=%s",
    self.base_url,
    self.model,
    len(self.slots),
    self.slots[^9_0].api_key[-6:] if self.slots else "none",
)
```

Then verify:

- If using Groq directly:
  - base URL should be `https://api.groq.com/openai/v1`, [\[104\]](#)
  - model must be a Groq-available model for your project, [\[102\]](#)
  - keys must be Groq keys.
- If using OpenRouter:
  - base URL should be `https://openrouter.ai/api/v1`, [\[file:query\]](#)
  - keys must be OpenRouter keys,
  - model should be an OpenRouter model ID.

## Direct answer

So: I do not think the main Groq issue is your round-robin code. I think the issue is that your router's configuration layer is inconsistent — the code is described as OpenRouter-backed, but the actual requests are going to Groq, and a 403 strongly suggests wrong key/base URL/model/project-permission alignment. [\[101\]](#) [\[102\]](#) [\[103\]](#)

Send me your config/settings.py next. That is the file most likely hiding the actual problem.



**GROQ\_API\_KEY=**

GROQ\_API\_KEY\_1=

GROQ\_API\_KEY\_2=

GROQ\_API\_KEY\_3=

GROQ\_API\_KEY\_4=

GROQ\_MODEL=llama-3.3-70b-versatile

LLM\_MODEL=llama-3.3-70b-versatile

LLM\_BASE\_URL=https://api.groq.com/openai/v1

this is the env structure rn

this is the config/settings.py

```
import os
```

```
from pathlib import Path
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
BASE_DIR = Path(file).resolve().parent.parent
```

```
DATA_DIR = BASE_DIR / "data"
```

```
OUTPUT_DIR = DATA_DIR / "outputs"
```

```
CHROMADB_DIR = DATA_DIR / "chromadb"
```

```
MCA_BULK_DIR = DATA_DIR / "mca_bulk"
```

```
RUN_LOG_PATH = DATA_DIR / "run_log.json"
```

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
CHROMADB_DIR.mkdir(parents=True, exist_ok=True)
```

```
MCA_BULK_DIR.mkdir(parents=True, exist_ok=True)
```

— Apollo.io — People + Company enrichment (legacy / optional) —————

**Free tier: 50 people searches/month + 50 company enrichments/month**

```
APOLLO_API_KEY = os.getenv("APOLLO_API_KEY", "")
```

## — SerpApi — Google Search/News discovery

---

Free tier: 100 searches/month. Get key at <https://serpapi.com>

Leave blank to fall back to DuckDuckGo HTML scraping (zero-cost).

```
SERPAPI_KEY = os.getenv("SERPAPI_KEY", "")
```

## — Hunter.io — Email finder

---

Free tier: 25 searches/month. Get key at <https://hunter.io/api-keys>

```
HUNTER_API_KEY = os.getenv("HUNTER_API_KEY", "")
```

— Skrapp.io — Email finder (free tier: 150 emails/month) —————

Get a key at <https://skrapp.io> → Settings → API. Leave blank to disable Skrapp.

```
SKRAPP_API_KEY = os.getenv("SKRAPP_API_KEY", "")
```

— LLM — Groq (OpenAI-compatible) with round-robin key pool —————

Primary source: GROQ\_API\_KEY / GROQ\_API\_KEY\_1..4

**Fallback: OPENAI\_API\_KEY /  
OPENAI\_API\_KEY\_1..4 (legacy)**

**LLM\_BASE\_URL defaults to Groq's OpenAI-compatible endpoint.**

```
_llm_key_candidates = [  
    os.getenv("GROQ_API_KEY", ""),  
    os.getenv("GROQ_API_KEY_1", ""),  
    os.getenv("GROQ_API_KEY_2", ""),  
    os.getenv("GROQ_API_KEY_3", ""),  
    os.getenv("GROQ_API_KEY_4", ""),  
    os.getenv("OPENAI_API_KEY", ""),  
    os.getenv("OPENAI_API_KEY_1", ""),  
    os.getenv("OPENAI_API_KEY_2", ""),  
    os.getenv("OPENAI_API_KEY_3", ""),  
    os.getenv("OPENAI_API_KEY_4", ""),  
]  
LLM_API_KEYS: list[str] = list(dict.fromkeys(k for k in _llm_key_candidates if k))  
LLM_MODEL = os.getenv("LLM_MODEL", "llama-3.3-70b-versatile")  
LLM_BASE_URL = os.getenv("LLM_BASE_URL", "https://api.groq.com/openai/v1")
```

**Legacy aliases — keep so any code that imports  
GROQ\_\* still works during transition**

```
GROQ_API_KEYS = LLM_API_KEYS  
GROQ_MODEL = LLM_MODEL
```

**— Jina AI Reader — converts any URL to clean  
markdown** 

---

**Optional key for token tracking. Supports multiple  
accounts for rotation.**

```
_jina_key_single = os.getenv("JINA_API_KEY", "")  
_jina_key_slots = [  
    os.getenv("JINA_API_KEY_1", ""),  
    os.getenv("JINA_API_KEY_2", ""),  
    os.getenv("JINA_API_KEY_3", ""),  
]  
JINA_API_KEYS: list[str] = list(dict.fromkeys(  
    k for k in ([_jina_key_single] + _jina_key_slots) if k  
))  
JINA_API_KEY = JINA_API_KEYS[0] if JINA_API_KEYS else ""
```

# Google Sheets

```
GOOGLE_SERVICE_ACCOUNT_JSON = os.getenv("GOOGLE_SERVICE_ACCOUNT_JSON", "")
GOOGLE_SHEET_ID              = os.getenv("GOOGLE_SHEET_ID", "")
GOOGLE_SHEET_TAB              = os.getenv("GOOGLE_SHEET_TAB", "Leads")
```

## Scraping

```
REQUEST_TIMEOUT = int(os.getenv("REQUEST_TIMEOUT", "15"))
MAX_RETRIES      = int(os.getenv("MAX_RETRIES", "3"))
PLAYWRIGHT_HEADLESS = os.getenv("PLAYWRIGHT_HEADLESS", "true").lower() == "true"
```

## Embedding model (runs locally, no API needed)

```
EMBEDDING_MODEL = "all-MiniLM-L6-v2"
CHROMA_COLLECTION = "leads"
```

## NLP

```
SPACY_MODEL = "en_core_web_sm"

LOOKBACK_DAYS_MANUAL = 30
LOOKBACK_DAYS_SCHEDULED = 7
```

Your `.env` and `settings.py` look **mostly correct**, so the Groq issue is now less likely to be the variable names and more likely to be one of these three things: **the API keys are actually empty/not loading, the keys are invalid/restricted, or Groq is rejecting requests from your current network/IP/project context.** Groq's docs say 403 means the request is denied due to permission restrictions, and model permissions can also be enforced at the org/project level. [\[119\]](#) [\[120\]](#) [\[121\]](#)

### What looks correct

These parts are fine:

- `LLM_BASE_URL=https://api.groq.com/openai/v1` matches Groq's OpenAI-compatible endpoint. [\[122\]](#) [\[123\]](#)
- `LLM_MODEL=llama-3.3-70b-versatile` is a valid Groq model name. [\[124\]](#) [\[123\]](#)
- `settings.py` correctly builds `LLM_API_KEYS` from `GROQ_API_KEY` through `GROQ_API_KEY_4`. `[file:query]`
- `llm_router.py` uses `OpenAI(api_key=..., base_url=...)`, which is the standard OpenAI-compatible pattern. [\[125\]](#)

So purely from code structure, nothing obvious is wrong there. [\[123\]](#) [\[125\]](#)

## Most likely issue now

### 1. The keys may actually be blank or not loaded

In the `.env` snippet you pasted, all the key lines are shown as:

```
GROQ_API_KEY=  
GROQ_API_KEY_1=  
GROQ_API_KEY_2=  
GROQ_API_KEY_3=  
GROQ_API_KEY_4=
```

If that is literally what is in your `.env`, then you have **no keys loaded**, and `router` should become `None` because `_build_router()` returns `None` when `LLM_API_KEYS` is empty. Your earlier logs, however, showed actual Groq requests being made, so either:

- you redacted the keys before sending them here, or
- the app is loading keys from somewhere else, or
- an old environment session still has them exported. [\[file:query\]](#) [\[126\]](#)

That means the first thing to verify is whether `load_dotenv()` is loading the same file you think it is loading. [\[file:query\]](#)

### 2. Invalid or unauthorized Groq key

If the keys are present but wrong, expired, copied incorrectly, or tied to a restricted project, Groq can return 403 permission errors. Groq's error docs explicitly say 403 is a permission restriction, and the official SDK maps 403 to `PermissionDeniedError`. [\[120\]](#) [\[119\]](#)

### 3. Network/IP restriction or environment issue

The Groq error message in your terminal said:

- Access denied. Please check your network settings. [\[126\]](#)

That wording strongly suggests Groq may be blocking your current network path, proxy, VPN, or hosting environment. There are also public reports of Groq 403s being tied to network/proxy context rather than code payload alone. [\[127\]](#) [\[128\]](#) [\[129\]](#) [\[126\]](#)

## What I think is *not* the issue

I do **not** think the main problem is:

- the model name `llama-3.3-70b-versatile`, because that model exists on Groq. [\[124\]](#)
- `LLM_BASE_URL`, because that endpoint is correct for Groq's OpenAI-compatible API. [\[122\]](#) [\[123\]](#)
- the round-robin design itself, because your requests are reaching the endpoint and failing with 403, which is downstream of routing. [\[119\]](#) [\[126\]](#)

## Most useful diagnosis

At this point, the highest-probability causes are:

| Likelihood | Cause                                | Why                                                                                                                    |
|------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| High       | Key is missing/not actually loaded   | Your pasted <code>.env</code> shows blank key values. <a href="#">[file:query]</a>                                     |
| High       | Key is invalid or unauthorized       | 403 from Groq usually means permission/auth restriction. <a href="#">[119]</a> <a href="#">[120]</a>                   |
| Medium     | Network/VPN/proxy issue              | Groq's message explicitly mentions network settings. <a href="#">[126]</a> <a href="#">[127]</a> <a href="#">[128]</a> |
| Medium     | Project/model permission restriction | Groq supports project/org model permissions. <a href="#">[121]</a>                                                     |
| Low        | Router logic bug                     | The request is reaching Groq; this is not a local control-flow failure. <a href="#">[126]</a>                          |

## What to test right now

Run a tiny standalone script outside your app. If this fails, the issue is not your router.

```
from openai import OpenAI
import os

client = OpenAI(
    api_key=os.getenv("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v1",
)

resp = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": "Say hello in one line."}],
    max_tokens=20,
)

print(resp.choices[0].message.content)
```

If this returns 403 too, then your issue is definitely:

- key,
- account/project permission,
- or network. [\[125\]](#) [\[119\]](#)

## Best next checks

Check these in order:

1. Print whether a key is actually loaded:

```
from config.settings import LLM_API_KEYS
print("keys loaded:", len(LLM_API_KEYS))
print("suffixes:", [k[-6:] for k in LLM_API_KEYS])
```

2. Print the exact runtime model and base URL:

```
from config.settings import LLM_MODEL, LLM_BASE_URL
print(LLM_MODEL, LLM_BASE_URL)
```

3. Test one key directly with the standalone script above.

4. If it still fails:

- try from a different network,
- disable VPN/proxy,
- generate a fresh Groq API key,
- confirm the project has access to llama-3.3-70b-versatile. [\[121\]](#) [\[120\]](#)

## One small code improvement

In `settings.py`, make `load_dotenv()` explicit so you know exactly which file is being read:

```
BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")
```

That avoids confusion if your app is launched from a different working directory. Your current `load_dotenv()` usually works, but being explicit is safer. [\[file:query\]](#)

## Direct answer

So, where do I think the Groq issue is?

**Most likely not in `llm_router.py` itself.**

I think the issue is in one of these:

- the actual `.env` key values being empty or not loaded, [\[file:query\]](#)
- the Groq keys being invalid/restricted, [\[120\]](#) [\[119\]](#)
- or Groq rejecting requests from your current network or project permissions. [\[127\]](#) [\[121\]](#) [\[126\]](#)

The next best file to inspect is not `llm_router.py` anymore — it's your **actual runtime key loading and a one-file standalone Groq test**. If you paste the output of `print(len(LLM_API_KEYS), LLM_MODEL, LLM_BASE_URL)` and the standalone test error, I can pinpoint it faster.

✱

# i want to make these agents work better tell me which codes you want i will send and you can revert with better codes

Yes — I can do that. The best way is for you to send me the files in a **specific order**, and I'll return improved versions one by one. From your structure, the biggest gains will come from fixing the **core infra layer first**, then the **orchestrator**, then the most important scrapers and enrichment agents. <sup>[137]</sup>

## Send first

These are the **highest-priority** files I want first:

1. `config/settings.py`
2. `utils/llm_router.py`
3. `utils/serp_client.py`
4. `utils/crawl4ai_fetch.py`
5. `utils/crawl4ai_fetcher.py`
6. `utils/http_fetch.py`
7. `main.py`
8. `agents/orchestrator.py`

Why these first:

- they control config, API usage, crawling, and execution flow,
- if these are weak, improving agents alone won't help much,
- your recent logs already showed Groq, SerpAPI, and crawl behavior issues around this layer. <sup>[138]</sup> <sup>[137]</sup>

## Then send agents

After that, send these agents in this order:

### Tier 1

- `agents/accelerator_scraper.py`
- `agents/job_scraper.py`
- `agents/enrichment_agent.py`
- `agents/mca_scraper.py`

These are most likely to benefit from better Crawl4AI usage, fallback logic, and normalization.

## Tier 2

- `agents/rss_scraper.py`
- `agents/gov_registry_scraper.py`
- `agents/icp_gate.py`
- `agents/parser_filter.py`
- `agents/scoring_agent.py`

These shape lead quality and filtering.

## Tier 3

- `agents/output_agent.py`
- `agents/verifier.py`
- `agents/compdata_scraper.py`
- `agents/razorpay_scraper.py`
- `agents/et_capital_scraper.py`
- `agents/llm_classifier.py`
- `agents/affordability_agent.py`
- `agents/apollo_enricher.py`
- `agents/skrapp_enricher.py`

These matter too, but they are less urgent than the core fetch/orchestration layer.

## Best workflow

Send files in **small batches**, not all at once.

### Batch 1

- `config/settings.py`
- `utils/llm_router.py`
- `utils/serp_client.py`
- `main.py`
- `agents/orchestrator.py`

### Batch 2

- `utils/crawl4ai_fetch.py`
- `utils/crawl4ai_fetcher.py`
- `utils/http_fetch.py`

## Batch 3

- `agents/accelerator_scraper.py`
- `agents/job_scraper.py`
- `agents/enrichment_agent.py`
- `agents/mca_scraper.py`

That will let me improve the architecture properly instead of patching isolated files.

## What I'll give back

For each batch, I can return:

- corrected code,
- better structure,
- cleaner async flow,
- stronger error handling,
- proper fallback behavior,
- better logging,
- and notes on what changed and why.

If you want, I can also make the code:

- **Crawl4AI-first**,
- **SerpAPI optional/fallback-safe**,
- **Groq-guarded** so the pipeline degrades gracefully when LLM calls fail.

## Most important files

If you want the **absolute minimum** to start, send just these 5 first:

- `agents/orchestrator.py`
- `agents/accelerator_scraper.py`
- `agents/job_scraper.py`
- `agents/enrichment_agent.py`
- `utils/crawl4ai_fetch.py`

That's enough for me to materially improve the agents layer.

## My recommendation

Start by sending this exact batch now:

- `config/settings.py`
- `utils/llm_router.py`

- `utils/serp_client.py`
- `utils/crawl4ai_fetch.py`
- `agents/orchestrator.py`

Once I see those, I'll tell you whether your agent problems are mostly:

- `infra/config`,
- crawl strategy,
- extraction design,
- or agent logic.

Send Batch 1 first, and I'll rewrite it cleanly.

✱✱

# I'll send you the agent files tell me what problems exists in them and then accordingly we will make them better

look at these first and then i'll the rest

Yes — these files already show a clear pattern of problems. The main issue is not just “bugs,” it’s that many agents are too **LLM-dependent for extraction**, weak on deterministic validation, and inconsistent in how they infer domains, dates, source URLs, and confidence. [\[139\]](#) [\[140\]](#) [\[141\]](#) [\[142\]](#) [\[143\]](#) [\[144\]](#)

## Main issues

Across the files, I see these recurring system-level problems: excessive use of LLMs for pages that should be parsed deterministically, weak normalization/dedup rules, fragile heuristics for company/domain inference, and several “fail-open” paths that can quietly let low-quality leads pass through. [\[140\]](#) [\[141\]](#) [\[145\]](#) [\[143\]](#) [\[144\]](#) [\[139\]](#)

There is also inconsistency between your newer Crawl4AI direction and older fetch/LLM patterns, so the codebase is halfway between two architectures right now. [\[141\]](#) [\[143\]](#) [\[139\]](#)

## File-by-file problems

| File                                | Biggest problems                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>accelerator_scraper.py</code> | Portfolio extraction is still mostly Groq-driven instead of schema-first; README/comments still describe “HTTP Reader → markdown → Groq” even though code partially moved to Crawl4AI; no strong post-extraction validation for company website, batch, or India relevance; duplicates are not normalized deeply. <a href="#">[139]</a> |
| <code>enrichment_agent.py</code>    | Domain guessing is risky; SMTP fail-open is dangerous because exceptions set <code>Ok = True</code> ; source-domain logic can misidentify true company domains; enrichment pipeline is doing too much inference from weak signals. <a href="#">[140]</a>                                                                                |

| File                                 | Biggest problems                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>gov_registry_scraper.py</code> | Government sources are being LLM-extracted from generic page markdown even when these sources are often tabular, form-based, or snapshot-based; <code>lookback_days</code> is barely enforced for some sources; snapshots only compare names, which can miss changed entities or formatting changes. <a href="#">[141]</a>            |
| <code>job_scraper.py</code>          | ICP filtering is substring-heavy and brittle; company career page domain is guessed as <code>cleanedcompanyname.com</code> , which will often be wrong; LinkedIn scraping is fragile against markup changes; no solid dedup across Wellfound/LinkedIn/career pages. <a href="#">[142]</a>                                             |
| <code>icp_gate.py</code>             | The LLM gate fails open by design when uncertain or when the LLM errors, which is good for recall but bad for a legal BD system; the prompt mixes hard constraints with soft heuristics, so consistency will be poor; no confidence score or evidence trail is stored. <a href="#">[145]</a>                                          |
| <code>mca_scraper.py</code>          | Zauba and fallback extraction are still LLM-heavy for registry pages; signal detection ignores event recency even though <code>lookback_days</code> is passed into <code>run</code> ; pacing with a global lock makes throughput poor; source URL is rebuilt from company name rather than actual matched page. <a href="#">[143]</a> |
| <code>llm_classifier.py</code>       | The classifier is too thin for production use; it accepts fuzzy partial matches of signal keys, which can misclassify; summary generation has no grounding guardrails beyond prompt text; no structured confidence or rationale. <a href="#">[144]</a>                                                                                |
| <code>apollo_enricher.py</code>      | Usage tracking is file-based and non-atomic; domain mismatch validation is too permissive because it returns valid with <code>suspicious_domain_mismatch</code> ; credit logic is practical but easy to desync under concurrency. <a href="#">[146]</a>                                                                               |
| <code>compdata_scraper.py</code>     | Nearly all extraction is delegated to the LLM even for directory-like pages; source-specific logic is minimal; <code>lookback_days</code> is unused; results lack confidence, evidence span, and deterministic filtering. <a href="#">[147]</a>                                                                                       |
| <code>et_capital_scraper.py</code>   | Batch LLM extraction over mixed article lists is high-risk for entity confusion; article dates are mostly replaced with <code>NOW</code> ; dedup is only by lowercased company name; RSS + listing extraction can collapse distinct rounds into one company record. <a href="#">[148]</a>                                             |

## Highest-risk bugs

A few issues are especially dangerous:

- In `enrichment_agent.py`, SMTP verification exceptions are treated as success with `ok = True`, so a network failure can promote a guessed email into an accepted email. [\[140\]](#)
- In `job_scraper.py`, watchlist career-page checks fabricate domains from company names, so many requests will hit the wrong site and may create false negatives or weird false positives. [\[142\]](#)
- In `mca_scraper.py`, `lookback_days` is not meaningfully applied to extracted corporate events, so stale MCA events can still become fresh lead signals. [\[143\]](#)
- In `icp_gate.py`, LLM failure or omission defaults to pass-through, which can flood downstream enrichment with junk. [\[145\]](#)
- In `apollo_enricher.py`, “domain mismatch” is treated as valid rather than reject-or-review, which weakens your strict email policy. [\[146\]](#)

## Design problems

Your current agents often use the LLM as the **primary extractor**, but for a Crawl4AI-first system it should usually be the **last-resort extractor** after deterministic selectors, regexes, table parsing, and schema extraction fail. [\[147\]](#) [\[139\]](#) [\[141\]](#) [\[143\]](#)

Also, most lead objects still miss a standard evidence model such as `evidence_text`, `event_date`, `extraction_method`, `confidence`, and `canonical_domain`, which makes dedup, verification, and debugging much harder. [\[148\]](#) [\[141\]](#) [\[143\]](#) [\[140\]](#)

## What I would fix first

Priority order should be:

1. **Stop unsafe acceptance paths** — fix SMTP fail-open, domain mismatch acceptance, and LLM pass-through behavior. [\[146\]](#) [\[145\]](#) [\[140\]](#)
2. **Standardize lead schema** across all agents. [\[139\]](#) [\[141\]](#) [\[142\]](#) [\[143\]](#)
3. **Replace LLM-first extraction with Crawl4AI schema-first extraction** for accelerator, compdata, job, MCA, and registry pages wherever possible. [\[147\]](#) [\[141\]](#) [\[142\]](#) [\[143\]](#) [\[139\]](#)
4. **Enforce event freshness** using extracted or source timestamps, not `datetime.now()`. [\[141\]](#) [\[148\]](#) [\[143\]](#)
5. **Add deterministic dedup/entity resolution** before ChromaDB-level semantic matching. [\[148\]](#) [\[142\]](#) [\[139\]](#)

## Best next step

Send the next batch, but before that, the most useful files now would be:

- `utils/crawl4ai_fetcher.py`
- `utils/http_fetch.py`
- `utils/llm_router.py`
- `utils/llm_json.py`
- `scoring_agent.py`
- `your orchestrator` or `main.py`
- any dedup / ChromaDB module

Then I can do the next level properly:

(1) identify exact code smells, (2) mark which parts should become Crawl4AI schemas, (3) tell you what to rewrite vs keep, and (4) give you a migration order file by file. [\[143\]](#) [\[139\]](#) [\[140\]](#)

If you want, I can next turn this into a strict checklist with three columns: **problem**, **why it matters**, **exact fix** for each file.

✱✱

# these are the remaining files

what changes are recommended here?

Yes — these files confirm that your current system is still **LLM-heavy in places where Crawl4AI can make it more deterministic**, and a few modules should be redesigned, not just patched. The biggest pattern is that extraction, validation, verification, and output are still shaped around “fetch text → ask LLM → clean up later,” while a Crawl4AI-first design should be “crawl with config → extract with schema → normalize → verify.” [\[150\]](#) [\[151\]](#) [\[152\]](#) [\[153\]](#)

## Highest-priority changes

I’d prioritize these modules first because they affect the whole pipeline:

| File                             | Recommended change                                                                                                                                                                                       | Why                                                                                                                                                         |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>orchestrator.py</code>     | Rewrite pipeline stages around <code>discovery</code> -<br>> <code>crawl</code> -> <code>extract</code> -> <code>normalize</code><br>-> <code>dedup</code> -> <code>verify</code> -> <code>output</code> | The current flow still mixes scraper behavior, enrichment, verification, and lead-volume guarantees in one orchestration path. <a href="#">[150]</a>        |
| <code>rss_scraper.py</code>      | Remove Groq headline extraction as the default path; use deterministic title parsing first, LLM only as fallback                                                                                         | The file currently sends RSS headlines to Groq just to extract company names, which is expensive and unnecessary for many feed items. <a href="#">[152]</a> |
| <code>razorpay_scraper.py</code> | Replace Playwright + giant extraction prompt with Crawl4AI page crawl plus CSS/schema extraction                                                                                                         | This scraper is exactly the kind of “listing page” Crawl4AI handles better with deterministic extraction. <a href="#">[151]</a>                             |
| <code>parser_filter.py</code>    | Convert from giant blacklist-heavy cleanup into a canonical normalization/validation layer                                                                                                               | Right now it is compensating for noisy upstream extraction rather than receiving cleaner source data. <a href="#">[154]</a>                                 |
| <code>verifier.py</code>         | Reduce LLM verification and make verification evidence-based per signal type                                                                                                                             | The verifier still relies on LLM judgment on search results for some claims. <a href="#">[153]</a>                                                          |
| <code>output_agent.py</code>     | Add stable IDs, evidence columns, and append-dedup for Sheets                                                                                                                                            | Output currently looks usable, but it lacks stronger idempotency and auditability. <a href="#">[155]</a>                                                    |

## Orchestrator

`orchestrator.py` is doing too much policy and too much cleanup after weak extraction. It runs multiple scraper families, then applies parser/filtering, scoring, company merge, RAG dedup, enrichment, internet verification, and even lead-count rules, which means orchestration is carrying business logic that should live in dedicated layers. [\[150\]](#)

Recommended changes:

- Split into explicit stages:
  - `discover_candidates()`
  - `crawl_and_extract()`

- `normalize_entities()`
- `deduplicate_entities()`
- `verify_signals()`
- `enrich_contacts()`
- `score_verified_leads()`
- `publish_outputs()`
- Remove “lead count guarantee” logic from the orchestrator, because forcing minimum output volume can bias quality and reintroduce junk. <sup>[150]</sup>
- Replace “RAG dedup first-class stage” with:
  1. exact name/domain dedup,
  2. normalized-name fuzzy dedup,
  3. embeddings only for ambiguous leftovers. The current flow gives ChromaDB a heavier role than necessary. <sup>[150]</sup>

## RSS scraper

The RSS scraper currently fetches feeds efficiently, but then it uses Groq to extract the company name from headlines, with regex as fallback. That is a sign the scraper is still built around LLM parsing instead of structured heuristics. <sup>[152]</sup>

Recommended changes:

- Keep RSS as a **discovery source**, not a final extraction source. RSS should produce candidate URLs, titles, dates, and provisional signals. <sup>[152]</sup>
- Do deterministic extraction first:
  - title pattern rules for “X raises...”, “X hires...”, “X launches...”
  - reject roundup/explainer/review/article patterns before any LLM call
- Only if the title is ambiguous, crawl the linked article with Crawl4AI and extract from the article page using filtered markdown plus a small schema or LLM fallback. <sup>[152]</sup>
- Add fields like:
  - `candidate_company_name`
  - `company_name_confidence`
  - `requires_article_crawl=True/False`
 so the orchestrator can decide whether to spend crawl/LLM effort. <sup>[152]</sup>

That one change alone should cut a lot of Groq usage. <sup>[152]</sup>

## Razorpay scraper

This file is a strong candidate for a full rewrite. Right now it fetches Razorpay pages, tries plain HTTP, falls back to JS rendering, then sends up to 12k characters into one big prompt asking for a JSON list of companies. That is brittle and expensive for a directory/listing source. <sup>[151]</sup>

Recommended changes:

- Replace `fetch_as_text(... js_render=True)` with a Crawl4AI crawler config for Razorpay pages. <sup>[151]</sup>
- Use CSS extraction for repeated page elements like company cards, links, logos, descriptions, and categories wherever possible. Crawl4AI supports schema-driven structured extraction, which is a better fit than a giant page-level prompt. <sup>[151]</sup>
- Use LLM extraction only for missing subfields like founder if the page mentions it irregularly.
- Store page-level metadata:
  - `crawl_url`
  - `crawl_timestamp`
  - `extraction_strategy = css / xpath / llm_fallback`
  - `evidence_snippet`
- Also reconsider whether `razorpay_rize_member` should be a signal or just a verification/supporting-evidence field; it currently boosts leads as a source-derived signal, but it may be better as a trust marker. <sup>[156] [153] [151]</sup>

## Parser/filter

`parser_filter.py` is doing a lot of defensive cleanup: HTML artifacts, cities, media jargon, government entities, headline fragments, long non-ICP lists, law firm patterns, and summary stripping. That tells me upstream extraction is too noisy. <sup>[154]</sup>

Recommended changes:

- Keep this module, but reposition it as:
  - `normalizer.py`
  - `entity_validator.py`
- Separate concerns:
  1. `normalize_company_name()`
  2. `validate_company_candidate()`
  3. `classify_icp_fit()`
  4. `clean_text_fields()`
- Move very large blocklists out of code into data files like:
  - `data/non_icp_companies.txt`

- `data/government_entities.txt`
- `data/law_firm_patterns.yml`
- Add canonicalization output:
  - `canonical_company_name`
  - `normalized_company_key`
  - `rejection_reason`
  - `validation_confidence`
- Right now the module mostly rejects bad names after the fact; with Crawl4AI-first extraction, it should become a final safety gate rather than a rescue layer. <sup>[154]</sup>

One small but important note: your `_HTML_ARTIFACT` pattern includes a plain space token in the regex, which makes it overly broad and can misclassify normal names. That should be fixed even before a larger refactor. <sup>[154]</sup>

## Scoring agent

The scoring file is cleaner than the scrapers, but it still mixes signal quality, affordability heuristics, and ICP blocking. It also awards bonuses using text heuristics from summary/keywords rather than verified evidence fields. <sup>[156]</sup>

Recommended changes:

- Score on **verified structured attributes**, not loose text whenever possible:
  - `funding_stage_verified`
  - `job_role_verified`
  - `company_exists_verified`
  - `sector_verified`
  - `contact_quality_score`
- Split scoring into components:
  - `signal_score`
  - `verification_score`
  - `contactability_score`
  - `fit_score`
- Remove ICP blocking from scoring and keep it in validation. Scoring should rank valid leads, not decide whether they are valid in the first place. <sup>[156] [154]</sup>
- Replace `matched_keywords` bonuses with evidence-driven bonuses, for example:
  - 2 corroborating funding sources
  - active job posting
  - official company website found
  - founder email on company domain

- Keep `apply_enrichment_bonuses()`, but rename it to something like `apply_contactability_score()` to reflect what it is actually doing. <sup>[156]</sup>

## Verifier

The verifier is closer to a good architecture because it already routes by signal type, but it still depends on SerpApi/DDG plus some LLM interpretation of search results. It also has a fairly permissive “assume active” pattern when checks fail. <sup>[153]</sup>

Recommended changes:

- Make verification source-aware:
  - registry/government sources → self-verified
  - accelerator/company listings → verified by source credibility + extraction evidence
  - news signals → corroborated by 2 sources when possible
  - job signals → page active + company match
- Add structured evidence fields on every verification result:
  - `verification_method`
  - `verification_sources`
  - `evidence_urls`
  - `evidence_snippets`
  - `verified_signal_type`
- For funding verification, do deterministic corroboration first:
  - count distinct domains
  - require company-name match
  - optionally require funding verb/round keyword match before using LLM adjudication. <sup>[153]</sup>
- For job verification, use Crawl4AI on the job page and verify:
  - page reachable
  - job title present
  - company name present
  - not closed/expired rather than treating crawl failure as soft-pass. <sup>[153]</sup>
- If you keep LLM in verification, use it only to resolve ambiguous evidence, not as the default judge.

## Output agent

The output layer is decent, but it should become more audit-friendly if this will be used by a law firm. Right now it writes verified/unverified/rejected CSVs and appends eligible verified rows to Google Sheets, which is useful but not fully idempotent. <sup>[155]</sup>

Recommended changes:

- Add immutable IDs:
  - `lead_id`
  - `company_id`
  - `signal_id`
- Add audit columns:
  - `canonical_company_name`
  - `normalized_company_key`
  - `verification_method`
  - `evidence_urls`
  - `crawl_timestamp`
  - `first_seen_at`
  - `last_seen_at`
- Change Sheets append behavior to **upsert/dedup** using `lead_id` or `company_id` + `signal_type` + `date`, otherwise repeated runs can duplicate rows. <sup>[155]</sup>
- Save a richer JSON file than just verified leads, ideally:
  - run metadata
  - verification breakdown
  - per-source counts
  - rejected reasons summary
- Keep the terminal summary, but also produce machine-readable run stats for dashboards. <sup>[155]</sup> <sup>[150]</sup>

## Skrapp enricher

This file is relatively small and fine structurally, but I'd downgrade its importance in the new design. It should be a late-stage enrichment option, not something central to the pipeline. <sup>[157]</sup>

Recommended changes:

- Use it only after:
  1. company website crawl,
  2. team/about/contact extraction,
  3. domain match validation.

- Record clearer provenance:
  - email\_source
  - email\_confidence
  - email\_domain\_match
- Add a guard so Skrapp is only called for leads that are already verified and high priority, which preserves the free quota. <sup>[157]</sup>

## New module structure

If you do the refactor, I'd suggest a structure like this:

- agents/discovery/
  - rss\_discovery.py
  - directory\_discovery.py
- agents/crawl/
  - crawl\_service.py
  - crawl\_configs/
- agents/extract/
  - css\_extractors.py
  - llm\_extractors.py
  - schemas/
- agents/normalize/
  - normalizer.py
  - entity\_resolution.py
- agents/verify/
  - signal\_verifier.py
  - funding\_verifier.py
  - job\_verifier.py
- agents/enrich/
  - contact\_enricher.py
  - skrapp\_enricher.py
- agents/output/
  - output\_agent.py
  - sheets\_writer.py

That would align much better with the logic already implied across your orchestrator, filter, verifier, and output files. <sup>[155] [154] [153] [150]</sup>

Best next step

The most useful next files from you would be the ones that still define the old fetch/LLM pattern, especially:

- `utils/http_fetch.py`
- `utils/llm_router.py`
- `utils/llm_json.py`
- any `accelerator_scraper.py`
- any `job_scraper.py`
- any `mca_scraper.py`
- any Chroma/vector-store code

Then I can give you a **file-by-file migration map to Crawl4AI**, including:

- what to delete,
- what to keep,
- what to rewrite,
- and what the new interfaces should look like. [\[151\]](#) [\[153\]](#) [\[150\]](#)

If you want, send `http_fetch.py`, `llm_router.py`, and one more scraper next — those three will reveal the exact refactor path.

✱✱

okay can you give the updated files at lease 5 files at a time?

Yes — that’s the right way to do it. I can give you **5 updated files per batch**, but to avoid breaking your project, I should do it in a dependency-safe order rather than dumping random rewrites. Based on the files you’ve shared, the first batch should target the modules that most directly shape crawl, normalization, orchestration, and output behavior. [\[158\]](#) [\[159\]](#) [\[160\]](#)

Recommended first batch

I suggest we do these **first 5 files**:

| Order | File                          | Why first                                                                                                              |
|-------|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1     | <code>orchestrator.py</code>  | This defines the end-to-end control flow and needs to match the new Crawl4AI-first architecture. <a href="#">[158]</a> |
| 2     | <code>rss_scraper.py</code>   | This is still Groq-heavy for headline parsing and should be made deterministic-first. <a href="#">[159]</a>            |
| 3     | <code>parser_filter.py</code> | It should become a cleaner normalization/validation layer instead of a rescue filter. <a href="#">[161]</a>            |
| 4     | <code>verifier.py</code>      | Verification must be aligned with the new evidence-first flow. <a href="#">[162]</a>                                   |

| Order | File                         | Why first                                                                                |
|-------|------------------------------|------------------------------------------------------------------------------------------|
| 5     | <code>output_agent.py</code> | Output should carry stable IDs, evidence, and better auditability. <a href="#">[160]</a> |

That is the best first batch because these five files affect almost every other module's interface, so updating them first gives the rest of the system a stable foundation. [\[161\]](#) [\[162\]](#) [\[158\]](#)

## Important constraint

I can write the updated code, but if I do it right now from only partial visibility, I'll have to assume the interfaces of missing modules like:

- `utils/http_fetch.py`
- `utils/llm_router.py`
- `rag/vector_store.py`
- `agents/enrichment_agent.py`
- `agents/icp_gate.py`
- `agents.affordability_agent.py`
- and any Crawl4AI wrapper you want to use

Those assumptions may cause import mismatches or force me to stub parts. That risk is especially high because your current `orchestrator.py` depends on several agents beyond the files already shared. [\[158\]](#)

## Best way to proceed

Send me these next, and I'll return the first 5 rewritten files in a usable form:

- `utils/http_fetch.py`
- `utils/llm_router.py`
- `rag/vector_store.py`
- `enrichment_agent.py`
- if you already created any Crawl4AI helper, send that too

If you want, I can still start immediately with the first batch using a **minimal-assumption refactor**, where I:

- preserve current function names where possible,
- reduce Groq dependence,
- make the code more modular,
- and add placeholders where your missing modules are still required.

## My recommendation

Choose one of these two options:

- **Option A: Safe refactor** — send the dependency files first, then I give you 5 production-ready updated files.
- **Option B: Fast refactor** — I start now and give you 5 rewritten files with a few clearly marked assumptions/stubs.

For your project, **Option A is better** because `orchestrator.py` is tightly coupled to several unseen modules. <sup>[158]</sup>

Reply with either:

- “**Start with Option B**” and I’ll begin rewriting the first 5 files now, or
- upload the dependency files for **Option A**.

\*  
\*\*

## keep the agents aside

this is our logs for now how do we fix the crawl4ai issues

Your Crawl4AI problem is **not one single bug**. The logs show three separate failure classes: **bad wait strategy**, **bad target URLs/domains**, and **using browser crawl on anti-bot pages where deterministic fallback should happen first**. The Groq rate limits are also amplifying the crawl failures by making every bad crawl more expensive. <sup>[163]</sup>

### Main failures

From the logs, the most obvious Crawl4AI failure is this repeated pattern: `Wait condition failed Timeout after 20000ms waiting for selector networkidle`, especially on Wellfound pages. That usually means your config is treating `networkidle` like a selector/wait target for pages that keep background requests open, so the browser never reaches the expected idle state within 20 seconds. <sup>[163]</sup>

The second failure is invalid or weak URL handling, shown by repeated `net::ERR_NAME_NOT_RESOLVED` for domains like `dream.com` and `lvlzerospotlight.com`, plus `net::ERR_EMPTY_RESPONSE` on some targets. That means your enrichment crawl is often trying guessed domains or unverified company URLs before checking whether the hostname actually resolves. <sup>[163]</sup>

The third failure is anti-bot friction on pages like Wellfound and some company sites, where Crawl4AI is clearly reaching the page layer but failing in a browser-driven path labeled `ANTIBOT` and `Proxy direct failed`. That suggests you need a **pre-crawl decision tree** instead of sending every URL through the same browser-heavy strategy. <sup>[163]</sup>

## What to fix first

Prioritize these changes in order:

1. **Remove `networkidle` as your default wait target** for job pages and dynamic sites. The logs show it timing out repeatedly on Wellfound, so use a lighter wait strategy such as `domcontentloaded` plus a real CSS selector for the result container, or a short fixed post-load delay. <sup>[163]</sup>
2. **Add URL validation before Crawl4AI starts**, because `dream.com` and similar failures are wasting crawl slots on domains that do not resolve. A DNS/HEAD precheck should reject bad domains before browser launch. <sup>[163]</sup>
3. **Split crawl modes by source type**:
  - RSS/article pages: plain HTTP or lightweight Crawl4AI.
  - Known structured pages: CSS/XPath extraction.
  - Heavy JS or anti-bot pages: browser mode only when necessary.
  - Bad/blocked pages: fallback to search-backed discovery or skip with reason. This matters because your current logs show browser crawl being used on many pages that either fail fast or do not need a browser at all. <sup>[163]</sup>
4. **Stop crawling guessed enrichment paths blindly** like `/about`, `/team`, `/contact`, `/about-us` for every company before validating the root domain. The logs show many sequential failures across all those paths for broken domains, which multiplies the cost of one bad website guess by five. <sup>[163]</sup>
5. **Reduce LLM calls during crawl troubleshooting**, because Groq is repeatedly hitting 429s while extraction is already weak for accelerator pages. That means the system is paying LLM cost on top of unstable crawling without getting reliable extraction back. <sup>[163]</sup>

## Concrete fixes

Here is the shortest practical fix list for Crawl4AI itself:

- **For Wellfound and similar pages**
  - Use `wait_until="domcontentloaded"` instead of waiting on `networkidle`. <sup>[163]</sup>
  - Set `wait_for` to an actual page selector only if you know it exists; otherwise leave it unset. <sup>[163]</sup>
  - Add a small `delay_before_return_html` like 2 to 4 seconds instead of relying on perpetual idle.
- **For enrichment/company pages**
  - First validate the root domain.
  - Only if the root responds, try `/about`, `/team`, `/contact`, `/about-us`.
  - Stop after the first 2 failures on the same domain.
- **For anti-bot pages**
  - Mark source as `browser_unstable`.

- Fall back to search snippets, RSS, sitemap, or alternate public evidence instead of retry looping in browser mode.
- **For extraction**
  - Use deterministic parsing first, LLM second.
  - Your accelerator logs show multiple successful Groq responses that still returned zero companies, so the issue is not just rate limits; the extraction route itself is brittle. <sup>[163]</sup>

## Source-specific diagnosis

A few parts of the logs are especially clear:

| Source/problem                                 | What logs suggest                                                                                    | Fix                                                                                                         |
|------------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Wellfound job pages                            | Repeated timeout waiting for <code>networkidle</code> under Crawl4AI anti-bot path. <sup>[163]</sup> | Use <code>domcontentloaded</code> , reduce wait complexity, add selector-based extraction only if verified. |
| Enrichment domains like <code>dream.com</code> | Domain guess is wrong or unresolved, causing <code>ERR_NAME_NOT_RESOLVED</code> . <sup>[163]</sup>   | Add domain resolution + canonical website discovery before crawling company subpaths.                       |
| <code>codeantai.com</code>                     | Browser reaches navigation step but gets <code>ERR_EMPTY_RESPONSE</code> . <sup>[163]</sup>          | Mark site unstable, retry once with lighter config, then skip and record failure reason.                    |
| Accelerator portfolio extraction               | Groq often returns no companies even after successful requests. <sup>[163]</sup>                     | Replace LLM-first extraction with CSS block extraction or markdown list extraction.                         |
| Groq usage                                     | Many 429s and retries across keys. <sup>[163]</sup>                                                  | Lower concurrency, cache results, and avoid calling Groq when crawl output is empty/low quality.            |

## Recommended guardrails

You should add these operational rules immediately:

- **Do not call Groq if Crawl4AI returned empty markdown, navigation error, or unresolved domain.** The logs show the system still spending LLM budget around unstable crawl stages. <sup>[163]</sup>
- **Cache successful page fetches by normalized URL** so repeated `/about`, `/team`, and root-page fetches do not happen across the same run. The logs show many repeated company-page fetch patterns. <sup>[163]</sup>
- **Tag every crawl result with failure type** such as:
  - `dns_failed`
  - `timeout_wait_condition`
  - `anti_bot_blocked`
  - `empty_response`
  - `selector_missing`

This will make your retry logic much smarter than generic exception retries.

- Cap per-domain crawl attempts to prevent one bad company website from consuming too much time.

## Best next code changes

Ignore the agents for a moment and patch the crawl layer with this order:

1. `crawl_url()` wrapper:

- validate URL
- resolve domain
- choose crawl mode
- catch Crawl4AI exception classes
- return structured result with status, error\_type, html/markdown, final\_url

2. `website_candidate_builder()`:

- derive canonical company website before trying subpaths
- only expand to /about, /team, /contact after root succeeds

3. `extract_if_useful()`:

- if markdown too short, skip LLM
- if selector extraction succeeds, skip LLM
- if anti-bot triggered, skip LLM and mark for alternate evidence path

4. `retry_policy`:

- retry timeouts once
- never retry DNS failures
- never retry repeated empty-response domains more than once

## What the logs say overall

Right now, Crawl4AI is **working technically on many normal sites** like Innovist, Upgrad, Licious, NoBroker, Shiprocket, FreshToHome, and Tavaga, where pages are fetched and completed successfully. The real problem is that your pipeline sends too many low-quality or hostile targets into the same crawl flow, so the failures look like a Crawl4AI issue when the bigger problem is crawl orchestration and target hygiene. <sup>[163]</sup>

Send me the file that currently wraps Crawl4AI, or the code inside your enrichment/job scraping that calls it, and I'll turn this into a **drop-in fix list with exact code edits**.



# tell me all the files that you need

To fix the Crawl4AI issues specifically, I do not need all agents first. I mainly need the files that define the crawl wrapper, URL discovery, wait strategy, retries, and the places where Crawl4AI is called on unstable targets like Wellfound and company /about pages. The logs already show timeouts on networkidle, anti-bot/browser failures, unresolved guessed domains, and repeated enrichment subpage crawling, so those are the files that matter most. <sup>[164]</sup> <sup>[165]</sup> <sup>[166]</sup>

## Files I need first

Please send these highest-priority files first:

| Priority | File                                                                             | Why I need it                                                                                                                                                                                                 |
|----------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1        | utils/crawl4ai_fetcher.py or whatever file contains crawlurl() / crawlurlplain() | This is the most important file because the logs point to wait-condition, browser-mode, and navigation handling problems coming from the Crawl4AI wrapper. <sup>[164]</sup> <sup>[165]</sup> <sup>[167]</sup> |
| 2        | utils/http_fetch.py                                                              | I need to compare your lightweight fetch fallback against Crawl4AI so I can route stable sources away from browser mode. <sup>[167]</sup> <sup>[165]</sup> <sup>[168]</sup>                                   |
| 3        | utils/serpclient.py                                                              | Your enrichment flow appears to guess websites and LinkedIn pages before crawling, so I need to see how canonical domains and fallback discovery are being chosen. <sup>[165]</sup> <sup>[168]</sup>          |
| 4        | config/settings.py                                                               | This likely contains timeout, headless, concurrency, and browser-related knobs that directly affect Crawl4AI behavior. <sup>[164]</sup> <sup>[167]</sup>                                                      |
| 5        | requirements.txt                                                                 | I need exact library versions because Crawl4AI behavior can change across versions, and your logs explicitly show Crawl4AI 0.9.0. <sup>[166]</sup>                                                            |

## Next batch

After that, send the files that call Crawl4AI most aggressively:

- job\_scraper.py — because Wellfound is repeatedly failing on the current wait strategy. <sup>[166]</sup> <sup>[164]</sup>
- enrichment\_agent.py — because it is crawling root plus /about, /team, /contact, and /about-us across guessed domains, which is causing many ERR\_NAME\_NOT\_RESOLVED and empty-response failures. <sup>[165]</sup> <sup>[166]</sup>
- accelerator\_scraper.py — because it uses Crawl4AI for portfolio pages and currently gets many empty extractions even after successful LLM calls. <sup>[167]</sup> <sup>[166]</sup>
- mca\_scraper.py — because the logs show Crawl4AI being triggered during the MCA watchlist pass. <sup>[166]</sup>
- verifier.py — because it should stop re-crawling dead or unstable job URLs in a costly way. <sup>[168]</sup>

## Optional but helpful

These are not mandatory for the first diagnosis, but they will help make the fix cleaner:

- `.env.example` with keys redacted
- any logging config file
- any custom retry/backoff helper
- any proxy configuration file
- any browser profile / Playwright setup file
- sample `run_log.json` from a failed run

## Minimum set

If you want the **smallest possible set**, send just these 5 first:

1. `utils/crawl4ai_fetcher.py`
2. `utils/http_fetch.py`
3. `utils/serpclient.py`
4. `config/settings.py`
5. `requirements.txt`

That will let me diagnose the root Crawl4AI problems around **waits, navigation, fallback routing, invalid domains, and retry policy** without needing the full agent layer. <sup>[164]</sup> <sup>[165]</sup> <sup>[166]</sup>

## Why these matter

Your logs already show:

- repeated `Timeout` after `20000ms` waiting for selector `networkidle` on `Wellfound`, which strongly points to bad wrapper config rather than agent logic, <sup>[166]</sup>
- repeated `net::ERR_NAME_NOT_RESOLVED` on guessed company domains, which points to website discovery and validation issues before Crawl4AI is called, <sup>[166]</sup>
- and repeated enrichment page fan-out over `/about`, `/team`, `/contact`, `/about-us`, which points to crawl orchestration inefficiency in the page-fetch layer. <sup>[165]</sup> <sup>[166]</sup>

Send those 5 first, and I'll give you a **Crawl4AI-only fix plan** with exact code-level changes.

✱

1. <https://hub.docker.com/r/unclecode/crawl4ai>
2. <https://www.youtube.com/watch?v=Ex3EpKxIMO0>
3. <https://github.com/topics/crawl4ai>
4. <https://docs.crawl4ai.com/assets/llm.txt/diagrams/extraction-llm.txt>
5. <https://decodo.com/blog/crawl4ai-tutorial>
6. <https://github.com/James-jinye/crawl4ai-->

7. <https://www.youtube.com/watch?v=M2T5c-Ps2ak>
8. <https://pypi.org/project/Crawl4AI/0.4.21/>
9. <https://docs.crawl4ai.com/core/cli/>
10. <https://docs.crawl4ai.com>
11. <https://www.youtube.com/watch?v=xo3qK6Hg9AA>
12. [https://huggingface.co/spaces/re-mind/Crawl4AI/blob/main/docs/examples/extraction\\_strategies\\_example.py](https://huggingface.co/spaces/re-mind/Crawl4AI/blob/main/docs/examples/extraction_strategies_example.py)
13. <https://github.com/kaymen99/ai-web-scraper>
14. <https://www.marktechpost.com/2026/04/14/a-coding-implementation-of-crawl4ai-for-web-crawling-markdown-generation-javascript-execution-and-llm-based-structured-extraction/>
15. [https://dev.to/ali\\_dz/crawl4ai-the-ultimate-guide-to-ai-ready-web-crawling-2620](https://dev.to/ali_dz/crawl4ai-the-ultimate-guide-to-ai-ready-web-crawling-2620)
16. image.jpg
17. image-2.jpg
18. image-3.jpg
19. <https://docs.crawl4ai.com/core/simple-crawling/>
20. <https://deepwiki.com/Akirazeor/crawl4ai/5.2-extraction-strategies>
21. <https://docs.crawl4ai.com/complete-sdk-reference/>
22. <https://deepwiki.com/unclecode/crawl4ai/5.2-css-and-regex-extraction>
23. [https://github.com/LLMFAO/crawl4ai/blob/main/docs/md\\_v3/tutorials/async-webcrawler-basics.md](https://github.com/LLMFAO/crawl4ai/blob/main/docs/md_v3/tutorials/async-webcrawler-basics.md)
24. [https://crawl.freec.asia/mkdocs/tutorial/episode\\_07\\_Content\\_Cleaning\\_and\\_Fit\\_Markdown/](https://crawl.freec.asia/mkdocs/tutorial/episode_07_Content_Cleaning_and_Fit_Markdown/)
25. <https://www.aidoczh.com/crawl4ai/core/markdown-generation/index.html>
26. <https://docs.crawl4ai.com/extraction/no-llm-strategies/>
27. [https://huggingface.co/spaces/re-mind/Crawl4AI/blob/main/docs/examples/extraction\\_strategies\\_example.py](https://huggingface.co/spaces/re-mind/Crawl4AI/blob/main/docs/examples/extraction_strategies_example.py)
28. image.jpg
29. image-2.jpg
30. image-3.jpg
31. <https://decodo.com/blog/crawl4ai-tutorial>
32. <https://gist.github.com/wyattowalsh/5447933f8b6c38cef9fa73f08ca8a979>
33. <https://www.bookstack.cn/read/crawl4ai-0.4-en/1fd95dd12887623e.md>
34. <https://github.com/unclecode/crawl4ai/discussions/448>
35. <https://docingest.com/docs/crawl4ai.com>
36. <https://www.scrapingbee.com/blog/crawl4ai/>
37. <https://thunderbit.com/blog/crawl4ai-review-and-alternative>
38. <https://hub.docker.com/r/unclecode/crawl4ai>
39. <https://www.youtube.com/watch?v=Ex3EpKxIMO0>
40. <https://knowledgesdk.com/compare/crawl4ai-vs-knowledgesdk>
41. <https://klymentiev.com/blog/groq-pricing>
42. <https://www.capturekit.dev/blog/4-best-scraper-serp-api>

43. <https://dataforseo.com/apis/serp-api/pricing>
44. <https://www.searchapi.io/pricing>
45. <https://prospeo.io/s/serpapi-pricing-reviews-pros-and-cons>
46. <https://www.grizzlypeaksoftware.com/articles/p/groq-api-free-tier-limits-in-2026-what-you-actually-get-uwysd6mb>
47. <https://www.getaiperks.com/en/ai/groq-free-tier-2026>
48. <https://trajectdata.com/serp/scale-serp-api/pricing/>
49. <https://www.buildmvpfast.com/tools/api-pricing-estimator/serpapi>
50. <https://serpapi.com/blog/breakdown-of-serpapis-subscriptions/>
51. <https://serpapi.com/enterprise>
52. <https://medium.com/serpapi/breakdown-of-serpapis-subscriptions-39213e52a952>
53. <https://serpapi.com/high-volume>
54. <https://serpapi.com/pricing>
55. [https://ai.riera.co.uk/tools/process\\_understanding/crawl4ai/](https://ai.riera.co.uk/tools/process_understanding/crawl4ai/)
56. <https://devtune.ai/verticals/web-data-infrastructure-for-ai/crawl4ai>
57. <https://console.groq.com/docs/rate-limits>
58. <https://serpapi.com/blog/breakdown-of-serpapis-subscriptions/>
59. <https://www.buildmvpfast.com/tools/api-pricing-estimator/serpapi>
60. <https://docs.crawl4ai.com/core/self-hosting/>
61. <https://www.talordata.com/blog/low-cost-google-serp-apis-2026>
62. <https://github.com/sadiuysal/crawl4ai-mcp-server>
63. <https://www.youtube.com/watch?v=MsxYoa-RwXY>
64. <https://prospeo.io/s/serpapi-pricing-reviews-pros-and-cons>
65. <https://www.youtube.com/watch?v=BEVRh5RfLrU>
66. <https://chatforest.com/reviews/crawl4ai-mcp-server/>
67. <https://thetoolshelf.dev/mcp/servers/crawl4ai-mcp/>
68. [https://www.youtube.com/watch?v=c5dw\\_jsGNBk](https://www.youtube.com/watch?v=c5dw_jsGNBk)
69. <https://console.groq.com/docs/rate-limits>
70. <https://serpapi.com/pricing>
71. [https://ai.riera.co.uk/tools/process\\_understanding/crawl4ai/](https://ai.riera.co.uk/tools/process_understanding/crawl4ai/)
72. <https://devtune.ai/verticals/web-data-infrastructure-for-ai/crawl4ai>
73. <https://dataforseo.com/apis/serp-api/pricing>
74. <https://dev.to/ritzaco/best-serp-api-comparison-2025-serpapi-vs-exa-vs-tavily-vs-scrapingdog-vs-scrapingbee-2jci>
75. <https://chatforest.com/reviews/crawl4ai-mcp-server/>
76. <https://thetoolshelf.dev/mcp/servers/crawl4ai-mcp/>
77. <https://www.everydev.ai/tools/crawl4ai>
78. [https://www.youtube.com/watch?v=c5dw\\_jsGNBk](https://www.youtube.com/watch?v=c5dw_jsGNBk)
79. <https://console.groq.com/docs/projects>

80. <https://serpapi.com/high-volume>
81. <https://skywork.ai/skypage/en/crawl4ai-mcp-server-live-web/1977970991117570048>
82. <https://www.talordata.com/blog/low-cost-google-serp-apis-2026>
83. <https://bestaeotools.com/tool/ai-overview-api-serpapi>
84. paste.txt
85. <https://stackoverflow.com/questions/68912059/module-not-found-error-when-using-google-search-api-serpapi>
86. <https://serpapi.com/integrations/python>
87. <https://github.com/serpapi/google-search-results-python>
88. <https://serpapi.com/search-api>
89. <https://pypi.org/project/google-search-results/1.8.2/>
90. paste.txt
91. <https://serpapi.com/blog/using-serpapi-in-google-colab/>
92. <https://serpapi.com/blog/scrape-google-lens/>
93. <https://serpapi.com>
94. [https://www.youtube.com/watch?v=-H9yN\\_jl4FQ](https://www.youtube.com/watch?v=-H9yN_jl4FQ)
95. <https://github.com/serpapi/serpapi-python>
96. <https://serpapi.com/blog/scrape-google-product-page-with-python/>
97. <https://medium.com/@chukhraiartur/using-google-trends-api-from-serpapi-980d648cfb38>
98. <https://kibana.briz.ua/blog/serpapi-python-effortless-google-search-1764804069>
99. <https://stackoverflow.com/questions/76801579/get-serpapi-search-results-into-csv-files-using-python>
100. <https://github.com/FrontAnalyticsInc/data-winners/blob/main/datasource-api-serpapi/serpapi-examples-for-google-search.ipynb>
101. <https://console.groq.com/docs/errors>
102. <https://console.groq.com/docs/model-permissions>
103. paste.txt
104. <https://theneuralbase.com/groq/qna/fix-groq-api-authentication-error/>
105. <https://github.com/RooCodeInc/Roo-Code/issues/5724>
106. <https://openrouter.ai/docs/api/reference/parameters>
107. <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
108. <https://learn.microsoft.com/en-ca/answers/questions/5838000/azure-openai-error>
109. <https://openrouter.ai/docs/sdks/python/api-reference/chat>
110. <https://theneuralbase.com/openai/errors/openai-permissiondeniederror-403-model-access/>
111. [https://dev.to/alice\\_kelly\\_68226d164218e/openai-compatible-base-url-troubleshooting-7-checks-before-you-blame-the-sdk-53ge](https://dev.to/alice_kelly_68226d164218e/openai-compatible-base-url-troubleshooting-7-checks-before-you-blame-the-sdk-53ge)
112. [https://docs.camel-ai.org/reference/camel.configs.openrouter\\_config](https://docs.camel-ai.org/reference/camel.configs.openrouter_config)
113. <https://stackoverflow.com/questions/38489386/how-to-fix-403-forbidden-errors-when-calling-apis-using-python-requests>
114. <https://help.openai.com/en/articles/5072518-controlling-the-length-of-openai-model-responses>

115. <https://github.com/langgenius/dify/issues/6114>

116. <https://ofox.ai/blog/openai-api-model-not-found-errors-troubleshooting/>

117. Screenshot-2026-06-20-123230.jpg

118. Screenshot-2026-06-20-123212.jpg

119. <https://console.groq.com/docs/errors>

120. <https://github.com/groq/groq-python>

121. <https://console.groq.com/docs/model-permissions>

122. <https://www.typingmind.com/guide/groq/llama-3.3-70b-versatile>

123. <https://pi.dev/models/groq/llama-3-3-70b-versatile>

124. <https://console.groq.com/docs/model/llama-3.3-70b-versatile>

125. <https://theneuralbase.com/groq/qna/fix-groq-api-authentication-error/>

126. paste.txt

127. <https://community.groq.com/latest?page=4>

128. <https://linux.do/t/topic/113317>

129. <https://github.com/langgenius/dify/issues/6114>

130. <https://docs.tambo.co/reference/llm-providers/groq>

131. <https://groqstatus.com/incidents/01K6XDYJ0ZVK2DN64SXD0TVY09>

132. <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct/discussions/113>

133. <https://huggingface.co/Groq/Llama-3-Groq-70B-Tool-Use>

134. <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct/discussions/69>

135. [https://www.reddit.com/r/LangChain/comments/1rmfn84/bizarre\\_403\\_forbidden\\_with\\_groq\\_api\\_langchain/](https://www.reddit.com/r/LangChain/comments/1rmfn84/bizarre_403_forbidden_with_groq_api_langchain/)

136. <https://inworld.ai/models/groq-llama-3-3-70b-versatile>

137. image.jpg

138. paste.txt

139. accelerator\_scraper.py

140. enrichment\_agent.py

141. gov\_registry\_scraper.py

142. job\_scraper.py

143. mca\_scraper.py

144. llm\_classifier.py

145. icp\_gate.py

146. apollo\_enricher.py

147. compdata\_scraper.py

148. et\_capital\_scraper.py

149. affordability\_agent.py

150. orchestrator.py

151. razorpay\_scraper.py

152. rss\_scraper.py  
153. verifier.py  
154. parser\_filter.py  
155. output\_agent.py  
156. scoring\_agent.py  
157. skrapp\_enricher.py  
158. orchestrator.py  
159. rss\_scraper.py  
160. output\_agent.py  
161. parser\_filter.py  
162. verifier.py  
163. paste.txt  
164. job\_scraper.py  
165. enrichment\_agent.py  
166. paste.txt  
167. accelerator\_scraper.py  
168. verifier.py